

University College London
Department of Computer Science



Augment, Prioritise, Learn

Data Augmentation and Curriculum Design for Robust Language-Model
Reasoning

Akshat Mittal

Degree programme: MSc Machine Learning

Supervised by: Dr. Ilija Bogunovic

This report is submitted as part requirement for the MSc Machine Learning at University College London. It is substantially the result of my own work except where explicitly indicated in the text.

The report may be freely copied and distributed provided the source is explicitly acknowledged.

September 2026

Abstract

Large language models have achieved strong results on mathematical reasoning and other verifiable tasks, yet their performance can remain sensitive to changes in problem presentation and the distribution of training data. This dissertation investigates how the construction and selection of data influence reasoning during evaluation and reinforcement-learning post-training.

We study two complementary approaches: constructing new training examples through data augmentation and selecting training examples through curriculum learning. First, we examine how different augmentations affect mathematical reasoning accuracy. We then test whether post-training on targeted augmentations can improve robustness, and how this interacts with a curriculum used to select problems. Finally, we adapt curriculum-based task selection to long-horizon terminal tasks to investigate whether it remains effective beyond mathematical reasoning.

Across 50 mathematical problems and four model-inference configurations, adding an intermediate reasoning step, an approach termed subproblem augmentation, was the most disruptive augmentation. In procedural arithmetic, training on this augmentation increased mean held-out accuracy from 18.48 to 28.40 per cent across two seeds under prioritised level replay (PLR), which prioritises tasks near the model’s current capability boundary. Gains extended to several external and perturbed benchmarks, although effects depended on the curriculum. In terminal tasks, PLR found more informative training groups but generalised less well than uniform sampling, achieving 34.0 versus 42.5 per cent completion on InterCode-Bash in a single training seed.

Overall, we found that robust post-training requires more than simply adding variation in the data or prioritising tasks that provide an immediate learning signal. Augmentations must target genuine weaknesses, while curricula must use model feedback without narrowing the range of capabilities being developed.

Acknowledgements

I would like to thank Prof. Dr. Ilija Bogunovic for giving me the opportunity to take my first meaningful steps into academic research, and for his invaluable mentorship and guidance throughout this project. Thank you to my collaborators Shyam Sundhar Ramesh, Robin Faro and Jue Du for their constant help and support throughout the project.

I am also grateful to Prof. Dr. Ilija Bogunovic for facilitating access to the University of Basel's computing resources. I thank the sciCORE team at the University of Basel for providing and maintaining the high-performance computing infrastructure used in this work. The computational work was carried out at sciCORE (<http://scicore.unibas.ch/>), the scientific computing core facility at University of Basel.

Finally, I would like to thank my family for their endless support.

Contents

Abstract

Acknowledgements

1	Introduction	1
1.1	Motivation and Research Problem	1
1.2	Objectives	2
1.3	Structure of This Thesis	3
1.4	Code	3
2	Literature Review	5
2.1	Verifiable Post-Training and the Role of Data	5
2.2	Data Augmentation and Robustness	6
2.3	Constructing Synthetic Verifiable Reasoning Data	8
2.4	Curriculum Learning and Unsupervised Environment Design	9
2.5	Synthetic Terminal Environments and Post-Training for Terminal Agents	11
3	Stress-Testing Mathematical Reasoning with Augmentations	13
3.1	Background and Motivation	13
3.2	Scope and Contributions	14
3.3	Experiment Setup and Methodology	14
3.3.1	Evaluation Data and Augmentations	14
3.3.2	Target Models	16
3.3.3	Evaluation and Inference Configuration	17
3.3.4	Evaluation Measures	18
3.4	Results and Analysis	18
3.4.1	Breakability Across Models and Augmentations	19
3.4.2	Comparison between Augmentations	20
3.4.3	Comparison between Models	20
3.4.4	Effect of Budgeted Thinking	21
3.4.5	Qualitative Analysis	21
3.5	Limitations	23
3.6	Conclusion	24
4	Augmentation-Aware Frontier Learning for Open-Ended Curricula	26
4.1	Background and Motivation	26
4.2	Scope and Contributions	27
4.3	Experiment Setup and Methodology	28

4.3.1	Training environment and curricula	28
4.3.2	Offline subproblem augmentation	31
4.3.3	Training configuration	33
4.3.4	Evaluation data	34
4.4	Results and Analysis	34
4.4.1	Primary comparison: PLR and Frontier Learning	34
4.4.2	Separating policy learning from curriculum feedback	36
4.4.3	Uniform and domain-randomisation baselines	37
4.4.4	Easy-ladder comparison of curriculum methods	38
4.4.5	External and perturbed evaluation	39
4.4.6	Interpretation	41
4.5	Ablation study: removing the curriculum	42
4.6	Limitations	43
4.7	Conclusion	44
5	Curriculum Learning for Terminal-Agent Post-Training	47
5.1	Background and Motivation	47
5.2	Scope and Contributions	48
5.3	Experimental Setup	49
5.3.1	Dataset	49
5.3.2	Terminal tasks and rewards	49
5.3.3	From Tasks to Levels	50
5.3.4	Model and execution environment	50
5.3.5	Experimental conditions	51
5.3.6	Training configuration	51
5.3.7	Evaluation protocol	52
5.4	Curriculum-Learning Algorithm	52
5.4.1	Prioritized level selection	53
5.4.2	Coverage and branching	53
5.4.3	Grouped rollouts and advantages	54
5.4.4	Asynchronous reinforcement learning	54
5.4.5	DPPO policy optimisation	54
5.4.6	Zero-signal batches	55
5.5	Results and Analysis	58
5.5.1	In-distribution performance	58
5.5.2	Out-of-distribution performance	59
5.5.3	Learning frontier and failure analysis	60
5.5.4	Overall interpretation	62
5.6	Limitations	62
5.7	Conclusion	63

6	Conclusion	66
A	Supplementary Material	68
A.1	Stress-Testing Mathematical Reasoning with Augmentations	68
A.1.1	Preliminary Experiments	68
A.1.2	Earlier Qwen2.5-72B Evaluation	68
A.1.3	Qwen3.5 Pilot Evaluations	68
A.1.4	Complete Evaluation Configuration	69
A.1.5	Augmentation Generation Prompts	70
A.1.6	Target-Model Evaluation Prompt	72
A.2	Augmentation-aware Frontier learning	73
A.2.1	Subproblem Family Catalogue	73
A.2.2	Reproducibility Record	73
A.3	Curriculum Learning for Terminal-Agent Post-Training	77
A.3.1	Terminal-Task Training Configuration	77
	Bibliography	80
	AI Usage Declaration	89

List of Tables

1.1	Structure of the Thesis	3
2.1	Approaches to constructing reasoning data	8
3.1	Mathematical augmentation types	15
3.2	Examples of augmented problems	16
3.3	Primary robustness evaluation settings	17
3.4	Break@10 across unchanged and augmented problems	19
3.5	Effect of budgeted thinking	21
4.1	Subproblem transformation groups	32
4.2	In-domain validation ladders	34
4.3	Hard-ladder accuracy for the primary curricula	35
4.4	Hard-ladder baseline accuracy	37
4.5	Easy-ladder accuracy	38
4.6	Clean external benchmark accuracy	40
4.7	Perturbed benchmark accuracy	40
5.1	Training configuration for Terminal tasks	51
5.2	TMax validation performance	59
5.3	Easy-task validation performance	59
5.4	InterCode-Bash performance	60
5.5	Post-coverage training diagnostics	60
A.1	Preliminary model evaluations for robustness	68
A.2	Preliminary results for Qwen3.5 robustness	69
A.3	Shared robustness evaluation settings	69
A.4	Cross-model inference settings	70
A.5	Qwen3-14B inference-policy ablation	70
A.6	Subproblem family catalogue	73
A.7	In-domain ladder configurations	74
A.8	Curriculum controller hyperparameters	74
A.9	Augmented Frontier Learning controller statistics	75
A.10	Model, rollout, and policy-optimisation configuration.	77
A.11	Curriculum, asynchronous-execution, and state-management configuration.	78

List of Figures

2.1	Model inconsistency under equivalent problem formulations	7
3.1	Mathematical stress-testing workflow	14
3.2	Break@10 by model and augmentation type	19
3.3	Background-augmentation reasoning trace	22
3.4	Subproblem-augmentation reasoning trace	22
3.5	Term-augmentation reasoning trace	23
4.1	Frontier Learning augmentation pipeline	29
4.2	Hard-ladder accuracy for PLR and Frontier Learning	35
4.3	Hard-ladder accuracy by level	36
4.4	Frontier Learning curriculum-feedback ablation	36
4.5	Comparison of curriculum and baseline methods	37
4.6	Hard-ladder learning curves	38
4.7	Easy-ladder curriculum comparison	39
4.8	External and perturbed effects of augmentation	41
4.9	Curriculum ablation under augmented training	42
4.10	Transfer gains over static-pool training	43
5.1	TMax success across native complexity axes	50
5.2	Primary terminal-task results	58
5.3	Informative groups during terminal-task training	61
5.4	InterCode-Bash trajectory outcomes	62
A.1	Curriculum-feedback controller trajectories	75

Chapter 1

Introduction

Modern large language models have become increasingly capable at mathematical reasoning, coding, and tool use. However, strong performance on established benchmarks does not always translate into reliable performance when tasks are presented differently. This dissertation therefore investigates language-model robustness under controlled perturbations and studies whether targeted data augmentation and adaptive curriculum design can improve the reliability and efficiency of post-training. This chapter motivates the research problem of this dissertation, states the research questions and hypotheses addressed, and outlines the structure of the thesis.

1.1 Motivation and Research Problem

Pre-training on internet-scale corpora gives large language models broad linguistic and factual knowledge, but many recent gains in multi-step reasoning have come from a subsequent “post-training” stage. Reinforcement learning, especially Reinforcement Learning from Verifiable Rewards (RLVR) has become a central approach within this stage of model development. RLVR is designed for domains where outcomes can be checked automatically, for example in mathematical reasoning, where answers can be compared with reference answers; coding, where programs can be executed against tests; and formal verification, where formal proofs can be checked by proof assistants. Systems such as DeepSeekMath and DeepSeek-R1 demonstrate that large-scale reinforcement learning can substantially improve mathematical reasoning (Shao et al., 2024; Guo et al., 2025). Related advances have reached gold-medal-standard performance at the International Mathematical Olympiad (Luong and Lockhart, 2025), while verifier-guided systems such as AlphaEvolve have found improved solutions to open problems in mathematics and computer science (Novikov et al., 2025).

These achievements are striking, but benchmark performance alone does not establish that a model will reason reliably and robustly. A model may solve a difficult problem in its familiar form and fail when irrelevant details, wording, numerical values, or intermediate steps are changed. The Robust Reasoning Benchmark finds that frontier models are more resilient to mathematical perturbations than open-weight models, although they remain vulnerable to some transformations. This issue is compounded in open-weight models. Robustness also varies across models and perturbation types. (Golikov et al., 2026). This is not only a mathematical issue. AgentNoiseBench shows that tool-using agents lose performance when user instructions or tool outputs contain controlled noise, even though

the underlying tasks remain solvable (Wang et al., 2026). Deployments in the real world are inherently noisy, and these results indicate that modern large language models may not be robust enough to be used reliably in real-world settings.

More training compute is not, by itself, a complete answer to this problem. Improving robustness may therefore require more than increasing the amount of post-training. It may also depend on how training problems are constructed and which problems are selected for each update. Data augmentation changes the problems available for training. It can expose a model to variations that preserve the underlying task while changing how it must be interpreted (Le et al., 2026). Curriculum learning can instead control which examples the model encounters, concentrating limited training compute on tasks that provide an appropriate learning signal at the model’s current level of competence (Bengio et al., 2009). These approaches are complementary: augmentation changes the experience available to the model, while a curriculum changes which experience is selected next.

The central research problem of this dissertation is therefore not simply whether models should receive more data. It is how training problems should be changed, selected, and ordered so that limited post-training compute produces reasoning that is both capable and robust. We study this problem first through controlled mathematical perturbations, then through augmentation inside a procedural frontier curriculum, and finally through curriculum selection of heterogeneous terminal tasks.

1.2 Objectives

The overarching objective of this dissertation is to understand how the construction and selection of training problems affect the robustness and efficiency of language-model post-training. It approaches this objective through three connected stages, moving from robustness evaluation to targeted augmentations at the frontier of a model’s capabilities, and then to curriculum learning in a more complex setting.

First, the dissertation evaluates robustness under controlled variation. It measures how reliably different language models solve transformations of mathematical problems and identifies which types of change expose the clearest failures. This establishes whether strong performance on the original questions remains stable when their presentation is altered.

Second, it studies augmentation within a learning curriculum. It tests whether training on targeted augmentations improves in-domain performance and out-of-distribution generalisation and robustness, and examines how these augmentations interact with curriculum-based approaches which select problems at the frontier of a model’s capabilities.

Finally, the dissertation extends adaptive task selection to terminal-agent training. It develops and evaluates a competence-aware curriculum for heterogeneous terminal tasks, with an emphasis on increasing useful learning signal per expensive interaction

and improving performance on held-out tasks. This tests whether the data-selection principles developed in procedural mathematics remain useful in a multi-turn, executable environment.

1.3 Structure of This Thesis

Chapter 2 reviews the work on which the dissertation builds, moving from verifiable post-training and robustness evaluation to data augmentation, procedural generation, and adaptive curricula. The three empirical chapters then progress from diagnosing sensitivity to changes in mathematical presentation, to training on such changes within a procedural curriculum, and finally to curriculum learning over a fixed collection of agentic environments. Table 1.1 summarises this progression.

Study	Setting and data intervention	Research role
Mathematical robustness Chapter 3	Curated mathematical problems; apply verified prompt transformations at evaluation	Identify which changes in presentation expose brittle reasoning.
Augmented frontier training Chapter 4	Procedurally generated decimal arithmetic; augment prompts while adaptively selecting difficulty levels	Test whether targeted exposure improves learning and robustness near the capability frontier.
Terminal-agent curricula Chapter 5	A finite, heterogeneous collection of executable terminal tasks; prioritise and replay existing environments	Test whether competence-aware selection improves reinforcement post-training beyond procedural mathematics.

Table 1.1: Progression and relationship of the three empirical studies.

1.4 Code

The code developed for the three empirical chapters are available in the following repositories:

- Chapter 3: <https://anonymous.4open.science/r/augmentation-reasoning-60FE>
- Chapter 5: <https://anonymous.4open.science/r/terminal-tasks-curriculum-4FF4>
- Chapter 4: <https://github.com/Rhine-AI-Lab/Accel-ver1> This Chapter’s code implementation incorporates this private research-group codebase and is not publicly released. Further implementation details may be provided on request, subject to the research group’s approval.

Chapter 2

Literature Review

This chapter reviews the research needed to understand how training data can be constructed and selected for robust language-model post-training. Its purpose is to establish the foundations for the three empirical chapters and to connect work on robustness, data augmentation, procedural generation, and curriculum learning.

It begins with verifiable post-training and the role played by the choice of training problems. It then examines robustness and data augmentation, followed by the main ways in which new reasoning problems can be constructed. The final two sections explain curriculum learning for generated problems and the additional challenges presented by terminal agents. Together, these areas address a common question: once model outputs can be checked automatically, which problems should a model learn from?

2.1 Verifiable Post-Training and the Role of Data

Post-training adapts a pretrained language model to particular behaviours or tasks. Supervised fine-tuning does this by teaching the model to imitate selected responses. Reinforcement learning instead lets the model attempt a problem and uses a reward to judge the result. Reinforcement learning with verifiable rewards (RLVR) is designed for tasks whose outcomes can be checked automatically (Shao et al., 2024; Guo et al., 2025). A mathematical answer can be compared with a known result, code can be run against tests, and the final state of a terminal environment can be inspected. This avoids requiring a person or a learned reward model to judge every attempt.

To see how a reward changes the model, let x be a prompt and $y = (y_1, \dots, y_T)$ a response sampled from a policy π_θ . Proximal Policy optimisation (PPO) reuses sampled responses for several updates while limiting how far the new policy moves from the policy that generated them (Schulman et al., 2017). For token t , define the probability ratio

$$q_t(\theta) = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}. \quad (2.1)$$

If $\hat{A}_t$ estimates whether the action was better or worse than expected, PPO maximises the clipped objective

$$J_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(q_t(\theta) \hat{A}_t, \text{clip}(q_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]. \quad (2.2)$$

The clipping term removes the incentive to change the probability ratio further in the advantage-improving direction once it exceeds the clipping range, but does not impose

a hard bound on policy changes. In standard PPO, $\hat{A}_t$ is obtained with the help of a learned value function. For large language models, training and storing this additional critic can be expensive.

Group Relative Policy optimisation (GRPO) removes the learned critic by comparing responses to the same prompt (Shao et al., 2024). Suppose the policy samples G responses and a verifier assigns rewards $r_1, \dots, r_G$. Standard GRPO uses the group to estimate the advantage of response i :

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \delta}, \quad \bar{r} = \frac{1}{G} \sum_{j=1}^G r_j, \quad (2.3)$$

where σ_r is the standard deviation of the group rewards and δ prevents division by zero. The response-level advantage is applied to its generated tokens. A simplified GRPO objective is

$$J_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min(q_{i,t}(\theta) \hat{A}_i, \text{clip}(q_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}). \quad (2.4)$$

Here $q_{i,t}$ is the PPO ratio from Equation 2.1 for token t of response i . The clipped term has the same role as in PPO, while the KL penalty discourages the policy from moving too far from a reference model. Implementations vary in whether they normalise the group rewards, how they estimate the KL term, and how they weight tokens. In each chapter, we state the variants used in this thesis. GRPO has become a common choice for mathematical post-training, including DeepSeekMath and DeepSeek-R1.

If all responses to a prompt are correct, there is little left to learn from it. If all responses are incorrect, the group supplies no successful response to reinforce. In either case, equal rewards give GRPO no relative signal. DAPO addresses this by removing groups with no reward variation (Yu et al., 2025). The most informative problems are often those the model can solve, but cannot yet solve reliably.

Automatic verification therefore solves only part of the data problem. It gives us a cheap way to quantify how an attempt ended, but not which problems should be created, varied, or attempted next. A first way to address this is data augmentation: obtain more evidence from a fixed problem by changing how it is presented.

2.2 Data Augmentation and Robustness

Data augmentation constructs additional examples by transforming existing ones. Early examples include elastic distortions for handwriting recognition (Simard et al., 2003). Data augmentation later became a standard part of deep learning: AlexNet used random crops, horizontal reflections, and changes to RGB intensities to specifically reduce overfitting and improve generalisation in computer vision tasks. (Krizhevsky et al., 2012)

In language tasks, transformations range from local edits, such as lexical replacement, to sentence-level paraphrasing and model-generated rewrites. (Chai et al., 2025). During training, augmentation can increase diversity, reduce reliance on superficial patterns, and improve data efficiency. During evaluation, it can test whether a prediction is stable under changes that should not affect the target.

Augmentation also provides a direct way to evaluate robustness. Standard benchmarks normally present each problem in one fixed form. A model can therefore obtain a high score while remaining sensitive to changes in wording, values, or surrounding information. Robustness evaluation tests the model on controlled variants of the same task. When a change preserves the problem, performance should remain stable.

Figure 2.1 gives a concrete example. Several models solve the original arithmetic word problem but fail after the same information is reordered or redundant information is added. The calculation has not changed but the failures arise from how the problem is presented.

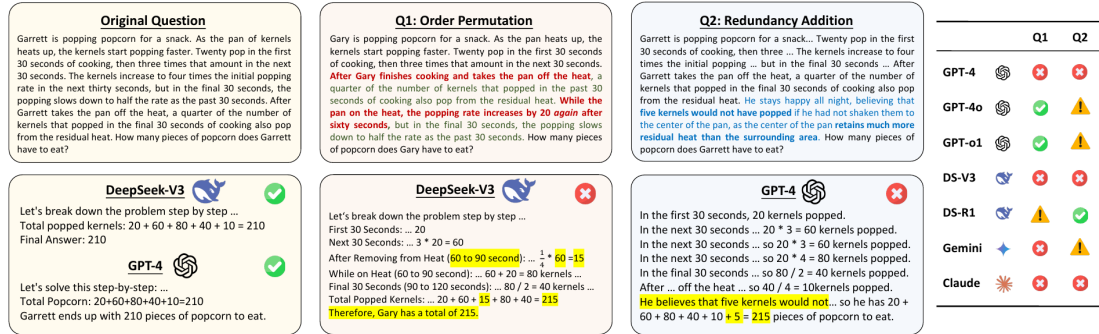


Figure 2.1: An original mathematical problem and two variants that reorder information or add redundant information. Several models answer the original problem correctly but fail on one or both meaning-preserving variants. Reproduced from (Yao et al., 2025), licensed under CC BY 4.0.

ReliableEval shows why evaluating only one formulation can be misleading (Lior et al., 2025). It samples meaning-preserving prompt formulations and reports a distribution of scores rather than a single value. Both the average score and its variation depend on the model and benchmark. Consequently, two models with similar scores under one formulation need not behave similarly over a broader set of valid formulations.

Recent work applies this approach specifically to mathematical reasoning. GSM-Symbolic varies the numbers inside familiar templates, while GSM-Plus changes wording, quantities, and question structure; both find that apparently small changes can cause large differences in accuracy (Mirzadeh et al., 2024; Li et al., 2024b). Template-based numerical tests trace these failures to both reasoning and calculation errors (Yang et al., 2025). PutnamGAP reaches the same broad conclusion on harder mathematics: across 18 models, mathematically equivalent variants remain more difficult than the original questions (Hao et al., 2025). The Robust Reasoning Benchmark applies a broader collection

of textual perturbations to AIME problems and reports that frontier models are generally more resilient than open-weight reasoning models, although neither group is uniformly robust (Golikov et al., 2026). Models are therefore not equally vulnerable to every transformation, but success on one presentation is not strong evidence of presentation-independent reasoning.

The same concern appears in tool-using agents. AgentNoiseBench introduces controlled noise into user instructions and tool outputs and reports performance degradation across a wide range of models (Wang et al., 2026). Robustness to variation is therefore relevant not only to static reasoning problems, but also to models that must act over several steps using information returned by external tools.

The mathematical robustness study in Chapter 3 uses augmentation in its diagnostic role. It compares answers across alternative presentations of the same underlying problem and distinguishes target-model failures from invalid transformations.

2.3 Constructing Synthetic Verifiable Reasoning Data

New reasoning data can be produced before training, generated by a model during training, or sampled from a procedural program. Table 2.1 sets out the main distinction. These approaches are not mutually exclusive: a procedural problem can subsequently be rewritten, and a model-generated variant can be checked by a program when its answer is known.

Table 2.1: Three approaches to constructing verifiable reasoning data.

Approach	How problems are obtained	Main trade-off
Offline synthesis	A model transforms or extends known problems before training.	Flexible generation, but the resulting training corpus is fixed.
Online synthesis	A model proposes new problems or variants while the policy is training.	Can respond to the current policy, but adds generation and validation cost to the training loop.
Procedural generation	A parameterised program produces both a problem and the information needed to verify its answer.	Fast and exactly checkable, but its coverage is determined by the programmed generator.

Offline mathematical synthesis usually begins with a known problem. A teacher model can rewrite it, make it harder, or produce a different solution path. WizardMath, MetaMath, and MuggleMath implement different versions of this approach, but all construct an expanded corpus before policy training begins (Luo et al., 2025; Yu et al., 2024; Li et al., 2024a). The corpus can be generated and inspected once, keeping teacher inference outside the training loop.

Other methods connect generation more closely to reinforcement learning. Transformation-Augmented GRPO generates equivalent rephrasings and pools evidence from the original and transformed questions, which can recover a useful update from a group that would

otherwise have uniform rewards (Le et al., 2026). MathForge makes questions harder through changes such as distracting context, abstract language, and nested subproblems, then gives greater weight to variants that remain challenging (Dai et al., 2026). Variational Problem Synthesis uses successful policy solutions to modify problems that remain difficult (Liang et al., 2025). These methods can change the training data as the policy improves, but generating and checking new questions during training increases cost.

Procedural content generation takes a different route. Instead of rewriting an individual example, a parameterised program defines a family of problems (Shaker et al., 2016). One setting of its parameters is commonly called a *level*. Repeatedly sampling a level produces different instances with the same structural properties and changing the parameters changes those properties or their difficulty. Since the program also computes the answer, it can supply a large stream of exactly verifiable problems without a teacher model.

Reasoning Gym applies this design to more than one hundred tasks across arithmetic, algebra, logic, graphs, algorithms, and games (Stojanovski et al., 2025). SCALER similarly provides verifiable environments with adjustable difficulty (Xu et al., 2026). The principle also extends beyond short reasoning questions. AlphaEvolve uses model-generated programs, automated evaluation, and evolutionary selection for algorithm discovery (Novikov et al., 2025), while Endless Terminals constructs task descriptions, executable environments, and tests for terminal agents (Gandhi et al., 2026). In each case, generation expands the available task space. It does not decide which part of that space should be sampled at a particular stage of training.

2.4 Curriculum Learning and Unsupervised Environment Design

Consider a generator that can create arithmetic expressions containing between two and twenty terms. A model may already solve most two-term expressions, while failing every attempt at the longest expressions. Both groups consume rollout compute but provide little relative reward signal. Intermediate configurations that are solved sometimes, but not reliably, are more likely to produce both successful and unsuccessful responses. A curriculum controls how frequently the model trains on each part of this difficulty range.

Curriculum learning originally described presenting examples in a useful order (Bengio et al., 2009). A fixed curriculum specifies that order in advance. An adaptive curriculum instead uses the learner’s recent performance to change what is sampled (Portelas et al., 2020). Unsupervised environment design extends this idea to the environments or levels themselves. PAIRED, for example, searches for environments that expose a performance gap between two agents and treats that gap as evidence of a challenging but feasible task (Dennis et al., 2020).

Prioritized Level Replay (PLR) stores previously encountered levels and revisits those

estimated to have high learning value (Jiang et al., 2021). In the original method, each level l_i has a score S_i derived from the magnitude of the Generalized Advantage Estimate on its latest trajectory. Rank-based prioritisation converts these scores into the replay distribution

$$P_S(l_i | \Lambda_{\text{seen}}, S) = \frac{h(S_i)^{1/\beta}}{\sum_j h(S_j)^{1/\beta}}, \quad h(S_i) = \frac{1}{\text{rank}(S_i)}, \quad (2.5)$$

where rank one denotes the highest-scoring level and β controls how strongly the ranking affects sampling.

A score becomes less reliable when its level has not been visited for some time. PLR therefore also defines the staleness distribution

$$P_C(l_i | \Lambda_{\text{seen}}, C, c) = \frac{c - C_i}{\sum_{C_j \in C} (c - C_j)}, \quad (2.6)$$

where c is the number of episodes sampled so far and C_i records when level l_i was last sampled. The final distribution in the PLR paper is

$$P_{\text{replay}}(l_i) = (1 - \rho)P_S(l_i | \Lambda_{\text{seen}}, S) + \rho P_C(l_i | \Lambda_{\text{seen}}, C, c). \quad (2.7)$$

Thus, replay favours levels with high estimated learning value while ensuring that older scores are eventually refreshed.

ACCEL adds mutation: a useful stored level can produce a nearby configuration by changing one or more of its parameters (Parker-Holder et al., 2022). Replay concentrates training on earlier experience, while mutation allows the set of levels to change with the learner. The quality of the priority signal affects how well the method performs. In particular, an imperfect regret measure may confuse a productive challenge with an impossible task (Rutherford et al., 2024).

Recent language-model curricula address the same selection problem at the level of prompts. Some adjust target difficulty using recent rewards (Zhao et al., 2024; Shi et al., 2025), others favour prompts whose rollout groups contain mixed outcomes or high success variance (Foster et al., 2025; Bae et al., 2025). GRESO predicts which prompts will have uniform outcomes so that some uninformative rollouts can be avoided entirely (Zheng et al., 2025). These methods select from supplied problems. Frontier Learning instead combines replay with a procedural generator: it estimates which generator levels lie near the policy’s current capability and explores or mutates levels to create new training problems there (Ramesh et al., 2026).

The thesis studies both curriculum regimes. Chapter 4 augments procedurally generated mathematics problems and tests the intervention under Uniform sampling, PLR, Domain randomisation, and a dynamic frontier curriculum. Chapter 5 instead applies curriculum learning to a large but finite collection of executable terminal environments.

2.5 Synthetic Terminal Environments and Post-Training for Terminal Agents

Terminal tasks make the data problem harder in several ways. Terminal agents must interpret a request, inspect and modify a changing software environment, recover from failures, and leave behind a state that satisfies an external verifier. Each task therefore requires not only a prompt, but also an executable environment, an initial state, dependencies, and tests. Terminal-Bench formalizes this setting using isolated environments and task-specific verification for realistic command-line workflows (Merrill et al., 2026).

Training recipes for these agents have progressed from prompting and execution scaffolds around pretrained models, to supervised fine-tuning (SFT) on successful interaction trajectories, and more recently to reinforcement learning directly in terminal environments. On the trajectory-data axis, Nemotron-Terminal studies synthetic task construction, filtering, curricula, and other data-engineering choices for terminal training (Pi et al., 2026), while TerminalTraj builds Docker-aligned tasks and verified trajectories from software repositories (Wu et al., 2026). These works show how executable, high-quality demonstrations can make terminal SFT effective at scale.

A complementary line scales the environments used for reinforcement learning. Endless Terminals autonomously constructs task descriptions, containers, completion tests, and solvability checks, then trains with binary outcome rewards (Gandhi et al., 2026). TMax uses a compositional taxonomy spanning difficulty, personas, and verifier design to generate a larger environment corpus, and trains agents with outcome-based reinforcement learning (Iverson et al., 2026). Chapter 5 studies curriculum learning for terminal-agent post-training.

Overall, the literature shows that automatic verification makes large-scale post-training possible, but does not decide which data a model should train on or guarantee robustness to variation. Augmentation, procedural generation, and curriculum learning address different parts of this problem, yet their interaction is not well understood. In particular, it remains unclear which mathematical transformations expose the clearest failures across models, whether training on those transformations improves generalisation, and whether curriculum signals remain useful when the presentation of a problem adds difficulty. The chapters that follow study these questions in turn.

Chapter 3

Stress-Testing Mathematical Reasoning with Augmentations

3.1 Background and Motivation

Mathematical reasoning benchmarks usually evaluate a model on one version of each problem. This tells us whether the model can solve the benchmark question as written, but not whether it can solve an equivalent question expressed in a different way. A model may, for example, answer a familiar formulation correctly but fail when the same mathematical information is placed in a new story or presented using unfamiliar terminology.

Exact-answer accuracy is a natural metric for mathematical reasoning: a response is correct if its final answer agrees with the reference answer. However, accuracy on the original prompt combines mathematical ability with sensitivity to the particular wording of the benchmark. Testing several equivalent versions of a problem provides a more detailed view of model behaviour. If performance is stable, the model appears to be responding to the underlying mathematical task. If performance varies substantially, the original benchmark score conceals a form of fragility (Lior et al., 2025; Yao et al., 2025).

The purpose of this study is therefore diagnostic. Rather than using augmentation to increase the amount of training data, we first use it to probe the neighbourhood around an existing problem. We evaluate the effect of five intended meaning-preserving augmentations on mathematical problem solving. The five transformations vary in how much they alter the prompt. Lexical substitutions and nuisance additions are relatively local, background and terminology change the presentation more substantially, and subproblem augmentation adds a compositional step to the task. We apply these augmentations to 50 problems from a local AIME-500 collection and compare the resulting performance of three language models.

Specifically, this chapter investigates:

1. which augmentations produce the largest reduction in mathematical performance
2. whether the same augmentations are difficult for different models
3. whether semantically richer, LLM-generated transformations are more challenging than programmatic surface changes.

We find that subproblem augmentation is the most difficult transformation for all three models. We also find that the three LLM-generated augmentations are more difficult

on average than the programmatic augmentations. These results motivate the use of subproblem augmentations in the procedural training study in the following chapter.

3.2 Scope and Contributions

The contribution of this chapter is threefold:

1. We conduct a mathematical augmentation study covering 50 source problems, five transformations, ten variants per transformation, and four inference configurations across three models.
2. We compare augmentation difficulty both within and across models, including a comparison between LLM-generated and programmatic transformations.
3. We use the observed robustness failures to select transformations for the augmentation-aware training experiments in the next chapter.

The augmentation framework itself was not developed as part of this contribution. The work presented here consists of the mathematical evaluation, its experimental execution, validation, and comparative analysis.

The remainder of this chapter is organised as follows. Section 3.3 describes the evaluation data, augmentations, models, inference configuration, and measures. Section 3.4 presents the quantitative and qualitative analysis. Section 3.5 discusses the limitations, and Section 3.6 summarises the findings and connects them to the next study.

3.3 Experiment Setup and Methodology

Figure 3.1 gives an overview of the evaluation workflow.

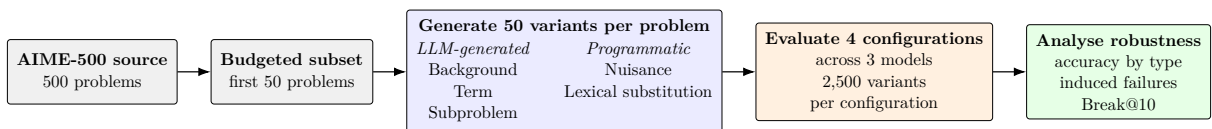


Figure 3.1: Overview of the mathematical stress-testing experiment. Each of the 50 source problems is transformed into ten variants of each of five types. The shared variant set is then evaluated using four inference configurations across three target models and analysed for robustness.

3.3.1 Evaluation Data and Augmentations

This section introduces the source problems and the transformations used to construct the evaluation set. Complete generation prompts and additional examples are provided in Appendix A.1.5.

Mathematical Problems

The source problems come from the AIME-500 collection released with ThinkBench (Huang et al., 2025b). AIME-500 contains 500 problems drawn from American Invitational Mathematics Examinations held between 1983 and 2023. We evaluate the first 50 entries. This subset was chosen because each source problem produces 50 augmented prompts, making evaluation substantially more expensive than a conventional single-prompt benchmark. In total, each inference configuration is evaluated on 2,500 variants.

Augmentation Types

Table 3.1 summarises the five augmentations. For every source problem, ten variants of each type are generated.

Family	Augmentation	Description
LLM-generated	Background	Places the mathematics in a new narrative setting
LLM-generated	Term	Introduces and defines a novel term
LLM-generated	Subproblem	Replaces a condition with a solvable subproblem
Programmatic	Nuisance	Adds irrelevant or distracting material
Programmatic	Lexical substitution	Replaces selected phrases with alternatives

Table 3.1: The five mathematical augmentation types.

Background augmentation. The problem is placed in a new story using objects and actions that differ from those in the source. The mathematical quantities and relationships are intended to remain the same. This tests whether a change of context affects the model’s ability to recover the underlying mathematics.

Term augmentation. A new abstract term is defined for a concept central to the problem, after which the question is restated using that term. The model must use a definition supplied within the prompt rather than relying only on familiar mathematical vocabulary.

Subproblem augmentation. A numerical condition is intended to be expressed through a second, self-contained mathematical problem whose answer supplies the required value. This transformation adds a compositional step: the model must interpret or solve the inserted subproblem before completing the original task.

Programmatic augmentations. Nuisance augmentation introduces irrelevant information and lexical substitution replaces selected words with intended low-risk alternatives. These transformations generally make more local changes than the three LLM-generated augmentations.

Original problem: Compute $\sqrt{(31)(30)(29)(28) + 1}$.		
Family	Type	Variant from the repository
LLM-generated	Background	In the ancient archives of a forgotten civilization, a historian discovers a clay tablet inscribed with a riddle: “To unlock the temple’s final seal, calculate the square root of the product of four consecutive integers—31, 30, 29, and 28—plus one.” What is the value required to open the seal?
LLM-generated	Term	Define the <i>quadravort</i> of an integer n as the value $\sqrt{n(n-1)(n-2)(n-3) + 1}$. Compute the quadravort of 31.
LLM-generated	Subproblem	Find the length of the hypotenuse of a right triangle whose legs have lengths 1 and $\sqrt{31 \times 30 \times 29 \times 28}$.
Programmatic	Nuisance	Compute $\sqrt{(31)(30)(29)(28) + 1}$. [73lmsmn1im5kpp]
Programmatic	Lexical substitution	Determine $\sqrt{(31)(30)(29)(28) + 1}$.

Table 3.2: One stored variant of every augmentation type for the same source problem. Mathematical text has been typeset for readability without changing its wording.

Variant Generation and Validation

Background, term, and subproblem variants were generated using Qwen3.5-72B (Qwen Team, 2026a) at a temperature of 0.7 with a maximum output length of 1,024 tokens. The prompt instructed the generator to preserve the answer and return only the transformed question. The shared generation template and the exact instruction used for each of these three augmentation types are reproduced in Appendix A.1.5. Nuisance and lexical variants were constructed programmatically according to the rules described in Section 3.3.1. The same variant pool was then presented to all three target models.

An augmentation prompt does not guarantee a valid transformation. Generated variants may accidentally change a mathematical condition, introduce ambiguity, or fail to apply the requested transformation. A representative sample of errored augmented questions is manually evaluated and presented in Section 3.4.5.

3.3.2 Target Models

The primary comparison includes Qwen3-72B (Qwen Team, 2025a), Qwen3-72B-Instruct-7207 (Qwen Team, 2025b), and Ministral-3-72B-Instruct-7212 (Mistral AI, 2026). These were the models for which evaluation was completed under a common protocol across all five transformations. The models differ in scale, model family, and post-training regime. Consequently, the comparison should be interpreted as a comparison of robustness across representative models rather than as a controlled scaling study.

Additional models were evaluated during preliminary experiments. Qwen2.5-72B-Instruct was evaluated on only four transformations using an earlier generation and

answer-extraction protocol, making its results unsuitable for direct inclusion in the primary comparison. Pilot evaluations were also conducted with Qwen3.5-0.8B and Qwen3.5-2B. Under the configurations tested, these models frequently produced long or repetitive reasoning traces and often failed to emit a parseable final answer within the configured generation budget. Because these runs did not satisfy the completion and protocol requirements of the primary experiment, they are reported separately in Appendix A.1.1.

3.3.3 Evaluation and Inference Configuration

All four configurations receive the same stored variants and use the same evaluation prompt and answer-scoring procedure. The exact system and user messages are reproduced in Appendix A.1.6. For each original problem, ten augmented problems are generated for each of the five augmentation types. This produces 50 augmented problems per original problem and 2,500 augmented problems in total. Each augmented problem is then evaluated once by each model–inference configuration. Each configuration also produces ten responses to every original problem for the clean Break@10 diagnostic described in Section 3.3.4. The principal settings are summarised in Table 3.3 and the complete run-specific configuration is provided in Appendix A.1.4.

Setting	Value
Source problems	50
Transformation types	5
Variants per type and source problem	10
Transformed prompts per configuration	2,500
Responses per transformed prompt	1
Additional clean diagnostic samples	10 per source problem
Required final-answer format	<code>\boxed{answer}</code>
Qwen3-14B budgeted-thinking reasoning budget	3,072 tokens
Qwen3-14B budgeted-thinking answer budget	128 tokens
Single-stage configurations’ output budget	16,384 tokens

Table 3.3: Principal settings used in the primary robustness evaluation.

Every target configuration is asked to solve the problem step by step and provide one final answer in `\boxed{}`. Qwen3-14B with thinking disabled, Qwen3-4B-Instruct-2507, and Ministral-3-3B-Instruct-2512 use single-stage greedy decoding for transformed prompts, with a maximum completion length of 16,384 tokens. The budgeted-thinking Qwen3-14B configuration instead uses sampling with the model-recommended thinking-mode settings and the bounded procedure described next.

Budgeted Thinking for Qwen3-14B

Qwen3-14B is evaluated with thinking enabled. Each response is allocated at most 3,072 tokens for reasoning. Generation stops early if the model emits the closing `</think>` delimiter. If the delimiter has not been produced when the reasoning budget is exhausted, the reasoning block is closed programmatically. The model is then allocated up to 128 additional tokens to produce a final answer in the required boxed format. This prevents an unfinished reasoning trace from consuming the complete output budget and ensures that every response has a separate opportunity to provide a final answer. The resulting configuration is referred to as *Qwen3-14B with budgeted thinking*.

The answer evaluator extracts a balanced boxed expression, with fallback handling for explicitly stated integer answers, and normalises LaTeX before performing numerical or symbolic comparison with the reference answer. We report the break rate.

3.3.4 Evaluation Measures

The primary result is estimated Break@10, the estimated probability that a set of ten evaluations of a source problem contains at least one failure. For source problem i , let q_i be the empirical correct-response proportion among its ten evaluations. We define

$$\text{Break}_i(10) = 1 - q_i^{10}, \quad \text{Break@10} = \frac{1}{50} \sum_{i=1}^{50} \text{Break}_i(10). \quad (3.1)$$

This is a macro-average of per-problem break probabilities, not the proportion of incorrect responses among the 500 evaluations. If k_i of the ten responses for problem i are correct, then $\hat{q}_i = k_i/10$ and its contribution is $1 - (k_i/10)^{10}$.

For the unchanged problem, the ten evaluations are stochastic generations of the same prompt at temperature 1.0. For an augmented problem, they are the ten distinct stored variants of the same source problem, each evaluated once. The two cases share the same evaluation budget and failure criterion, but the source of variation differs: decoding randomness for the original prompt and prompt transformation for the augmented conditions. We therefore describe them as *sampling Break@10* and *transformation Break@10*, respectively.

Additionally, we report the answer-found rate. It is a separate output-format metric: it records whether a response contains an extractable final answer. Exact-answer accuracy is retained as a secondary description of performance. Confidence intervals are calculated by bootstrap resampling over the 50 source problems.

3.4 Results and Analysis

This section first compares sampling and transformation Break@10. It then contrasts the two augmentation families and the four model-inference configurations. Lower values

indicate greater stability under the corresponding set of ten evaluations.

3.4.1 Breakability Across Models and Augmentations

Table 3.4 reports Break@10 for the unchanged prompts and each augmentation type. The lowest value in each row is shown in bold.

Condition	Qwen3-14B no thinking	Qwen3-14B budgeted	Qwen3-4B-Instruct	Ministral-3B
Original	89.7	45.1	55.5	99.7
Background	89.4	70.5	61.3	92.9
Term	84.2	66.0	67.6	89.0
Subproblem	97.8	99.2	96.8	98.2
Nuisance	71.8	47.4	44.8	78.2
Lexical substitution	73.2	44.4	41.0	76.3

Table 3.4: Macro-averaged estimated Break@10 (%) as defined in Section 3.3.4. The 500 binary outcomes in each condition are not pooled into a single error rate. The break probability is computed separately for each source problem and then averaged over the 50 problems.

Figure 3.2 presents the same results grouped by model, making it easier to compare the pattern of transformation Break@10 within each model–inference configuration.

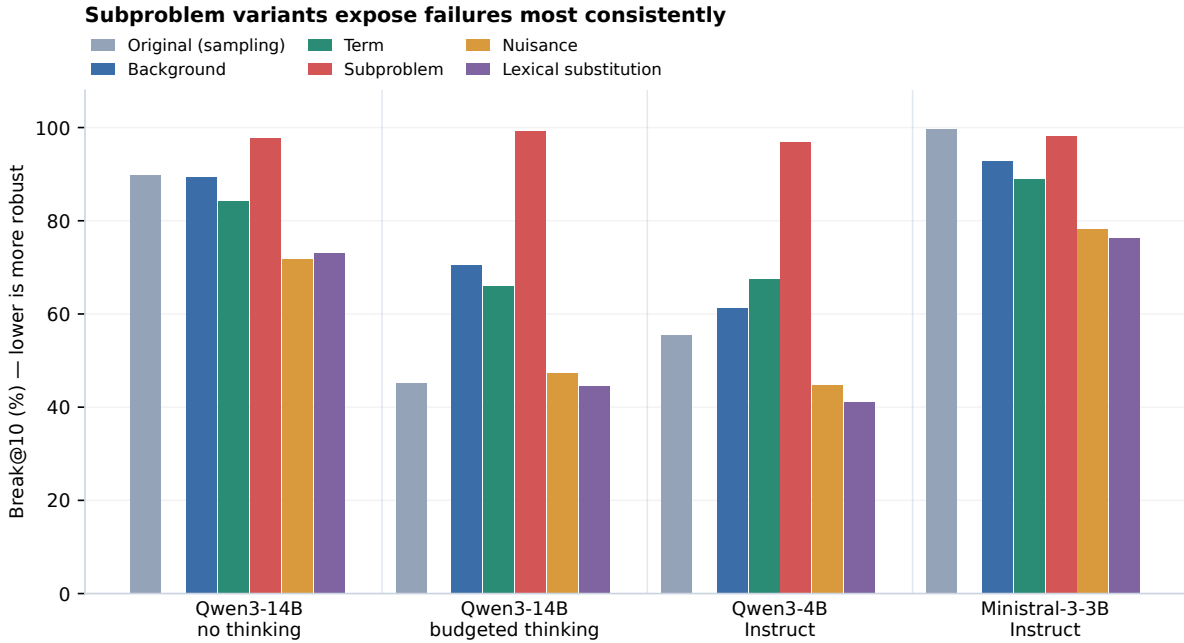


Figure 3.2: Break@10 by model and augmentation type. Within each model group, the grey bar reports sampling Break@10 on unchanged prompts, while the coloured bars report transformation Break@10 for the five augmentation types. The gap separates the two sources of variation; they should not be interpreted as a direct before-and-after comparison. Lower is better.

The original and transformed bars should not be treated as a before-and-after comparison. The original condition measures stochastic decoding at temperature 1.0, whereas transformed prompts are decoded greedily. Lower values for nuisance and lexical substitution therefore do not show that these transformations improve reasoning. Break@10 is also lower when failures are concentrated on a few consistently difficult problems rather than spread across many problems. In addition, some reformulations may be easier for a particular model.

3.4.2 Comparison between Augmentations

Subproblem augmentation has the highest transformation Break@10 for every configuration, ranging from 96.8% to 99.2%. Background and term augmentation also have higher Break@10 than each programmatic transformation for every configuration. The relative order of background and term is not universal: background is more breakable for both Qwen3-14B configurations and Ministral-3B, whereas term is more breakable for Qwen3-4B-Instruct.

Among the programmatic transformations, lexical substitution has the lowest Break@10 for budgeted Qwen3-14B, Qwen3-4B-Instruct, and Ministral-3B; nuisance is lowest for Qwen3-14B without thinking. Their Break@10 values nevertheless range from 41.0% to 78.2%, so these transformations still expose failures for a substantial proportion of source problems. Overall, subproblem augmentation is the most reliable stress test, while nuisance and lexical substitution provide weaker but more model-dependent tests of surface-form robustness.

3.4.3 Comparison between Models

On the original problems, budgeted Qwen3-14B has the lowest sampling Break@10, at 45.1%, followed by Qwen3-4B-Instruct at 55.5%. Qwen3-14B without thinking is considerably more breakable at 89.7%, while Ministral-3B has the highest value at 99.7%. Thus, under temperature-1.0 stochastic decoding, budgeted Qwen3-14B is the most stable configuration on the unchanged prompts. The difference between the two Qwen3-14B results also indicates that inference policy can materially affect the probability of exposing a failure even when the checkpoint is held fixed.

The Qwen3-4B-Instruct configuration has the lowest Break@10 for background, subproblem, nuisance, and lexical-substitution augmentation. Budgeted Qwen3-14B is lowest for term augmentation, at 66.0% compared with 67.6% for Qwen3-4B-Instruct. These two configurations are also close on the two programmatic transformations: their values differ by 2.6 points for nuisance and 3.4 points for lexical substitution. Both are substantially less breakable on these local transformations than Qwen3-14B without thinking and Ministral-3B.

Subproblem augmentation provides little separation between models because every value is close to 100%. The original and transformed values should still be compared with caution because their variation comes from stochastic decoding and distinct prompt transformations, respectively.

3.4.4 Effect of Budgeted Thinking

The two Qwen3-14B configurations use the same checkpoint, source problems and transformations, but different inference policies. Table 3.5 shows that budgeted thinking reduces Break@10 for four of the five transformations. The reduction is largest for lexical substitution at 28.7 percentage points. In contrast, subproblem Break@10 increases slightly and the confidence interval for the change includes zero. Bounded reasoning therefore makes failures harder to expose for most transformations, but provides no clear evidence of improving robustness to the inserted compositional task.

Augmentation	No thinking	Budgeted thinking	Paired change [95% CI]
Background	89.4	70.5	−18.8 [−29.4, −8.9]
Term	84.2	66.0	−18.2 [−27.9, −9.5]
Subproblem	97.8	99.2	+1.4 [−1.1, +5.7]
Nuisance	71.8	47.4	−24.4 [−35.9, −13.8]
Lexical substitution	73.2	44.4	−28.7 [−41.7, −15.8]

Table 3.5: Effect of budgeted thinking on Qwen3-14B transformation Break@10 (%). Negative changes indicate improved stability. Confidence intervals are obtained by paired bootstrap resampling of the 50 source problems.

3.4.5 Qualitative Analysis

Break@10 shows how readily an evaluation budget exposes an incorrect answer, but it does not establish why that answer occurred. In particular, an inherited source answer can mark a response as incorrect even when an augmentation has changed the mathematical question. We therefore conducted a targeted response-level follow-up using Qwen3-4B-Instruct and inspected whether selected variants retained the source task.

Selection and Verification

Candidate cases were drawn from variants that had previously received an incorrect greedy answer. Each candidate was paired with a fresh evaluation of its source problem, and it was retained only when the source response was correct, the variant response was incorrect, and manual inspection found the mathematical task unchanged. The follow-up used the same Qwen3-4B checkpoint, system prompt, chat template, greedy decoding and token limit as the primary transformed-prompt evaluation.

Observed Failure Modes

Figures 3.3–3.5 show three representative failures. In each case, the left-hand trace is a correct response to the source question and the right-hand trace is an incorrect response to its augmented version. All three variants preserve the source problem.

SOURCE: CORRECT RESPONSE	AUGMENTED: INCORRECT RESPONSE
ORIGINAL QUESTION For how many positive-integer pairs (x, y) with $y < x \leq 100$ are both x/y and $(x + 1)/(y + 1)$ integers? MODEL REASONING “Count the total number of such (k, d) pairs.” ... “73 + 4 = 77; 77 + 3 = 80; 80 + 2 = 82; 82 + 1 = 83; 83 + 1 = 84; 84 + 1 = 85.” “Total = 85.”	AUGMENTED QUESTION A weaver uses positive integer thread counts x and y, with $y < x \leq 100$. The ratios x/y and $(x + 1)/(y + 1)$ must both be whole numbers. How many pairs give a stable pattern? MODEL REASONING $x = ky$ and $\frac{ky + 1}{y + 1} = k + \frac{1 - k}{y + 1}$, so $y + 1 \mid k - 1$. ... “Singles: 39, 40, 41, 45, ..., 99 → 30 values.” “30 singles + 3 doubles = 36; 48 + 36 = 84.”

Figure 3.3: Background augmentation: the source response completes the tally correctly, while the augmented response counts a list of 31 values as containing 30. Questions are shortened for layout and ellipses mark omitted response text.

The background example shows a bookkeeping failure rather than a misunderstanding of the story. The response removes the tapestry context, writes $x = ky$, and correctly obtains the condition $y + 1 \mid k - 1$. Its final list contains 31 values of k that contribute one solution, but the response counts only 30 and therefore reports 84 rather than 85.

SOURCE: CORRECT RESPONSE	AUGMENTED: INCORRECT RESPONSE
ORIGINAL QUESTION How many 15-toss sequences contain exactly two HH, three HT, four TH, and five TT adjacent subsequences? MODEL REASONING “The run sequence must be T, H, T, H, T, H, T, H.” “For T-runs: $\binom{8}{3} = 56$. For H-runs: $\binom{5}{3} = 10$.” “Therefore, $56 \times 10 = \span style="border: 1px solid black; padding: 0 2px;">560.”$	AUGMENTED QUESTION A length-14 path on vertices H and T has exactly two $H \rightarrow H$, three $H \rightarrow T$, four $T \rightarrow H$, and five $T \rightarrow T$ edges. How many vertex sequences have these counts? MODEL REASONING “We can now count the number of subsets $S \subseteq \{1, 2, \dots, 12\}$, size 5, such that ...” “This seems complex, but perhaps symmetry or a known result.” “After research or experience, in such problems, the number of such sequences is 840.”

Figure 3.4: Subproblem augmentation: the source response completes a stars-and-bars count, while the augmented response abandons its subset formulation and states 840 without deriving it.

The subproblem example preserves the four transition counts by representing a 15-toss sequence as a length-14 walk on vertices H and T . The response correctly notes that

the imbalance between HT and TH fixes the endpoints and begins a run-based count. It then reformulates the remaining task as a constrained subset count, describes that count as complex, and gives 840 without completing it. Here the failure is an unsupported conclusion after an unfinished combinatorial derivation, rather than a semantic defect in the transformed question.

SOURCE: CORRECT RESPONSE	AUGMENTED: INCORRECT RESPONSE
ORIGINAL QUESTION Given $\log_x w = 24$, $\log_y w = 40$, and $\log_{xyz} w = 12$, find $\log_z w$. MODEL REASONING Let $W = \log w$, $X = \log x$, $Y = \log y$, and $Z = \log z$. Then $W = 24X$, $W = 40Y$, and $W/(X + Y + Z) = 12$. <math>“Z = \frac{2}{3}Y”, so $\log_z w = \frac{W}{Z} = \frac{40Y}{(2/3)Y} = \boxed{60}.$</math>	AUGMENTED QUESTION The <i>logarithmic flux</i> of base b with respect to w is $\log_b w$. The fluxes of x, y, and xyz are 24, 40, and 12. Find the flux of z. MODEL REASONING <math>“The logarithmic flux of x is 24 $\rightarrow \log_b x = 24.”$ $“\log_b(xyz) = \log_b x + \log_b y + \log_b z.”$ <math>“12 = 24 + 40 + \log_b z, so $\log_b z = \boxed{-52}.”$</math></math>

Figure 3.5: Term augmentation: the source response keeps the logarithm’s base and argument in their correct roles, while the augmented response reverses them when applying the newly defined term.

The final example shows propagating errors. The augmented response reverses the logarithm’s base and argument and consequently applies the logarithm rules incorrectly.

3.5 Limitations

The experiment provides a detailed evaluation of multiple formulations per source problem, but several limitations constrain the conclusions.

First, the study uses the first 50 entries of the local AIME-500 collection rather than a random or complete sample. The results may therefore depend on the difficulty and topic distribution of this subset. Repeating the analysis on a larger random sample would provide stronger evidence of generalisation.

Second, transformations are intended to preserve mathematical meaning, but this cannot be assumed automatically. The manual audit is necessary to distinguish failures of the target model from invalid generated questions.

Third, the saved clean diagnostic uses a different decoding protocol from the augmented evaluation. This does not invalidate comparisons among augmented conditions, which all use similar protocols, but it prevents a valid calculation of degradation from the clean prompt until the matched evaluation is run.

Finally, the three models do not provide a controlled comparison of model size. The generator model, augmentation prompts, answer extractor, and large output-token budget are additional sources of experimental dependence.

3.6 Conclusion

This chapter examined whether mathematical performance remains stable when the same underlying problem is presented through different augmentations. Across all four model–inference configurations, subproblem augmentation had the highest transformation Break@10. LLM-generated background, term, and subproblem transformations were also more breakable on average than nuisance and lexical transformations.

These results show that solving a mathematical problem once does not guarantee that a model can solve the same problem when it is presented differently. Augmentations can therefore reveal weaknesses that remain hidden when evaluation uses only the original wording. They also demonstrate that not all augmentations affect the break rate of all the models equally. Subproblem augmentation was the strongest stress test in this study.

The next chapter asks whether training on such augmented problems can reduce this brittleness. It introduces augmentation into a procedural training pipeline and studies it alongside methods that select problems near the model’s current level of ability. The goal is to test whether augmented training improves robustness without harming performance on clean problems, and whether its effect depends on how training problems are selected.

Chapter 4

Augmentation-Aware Frontier Learning for Open-Ended Curricula

4.1 Background and Motivation

The previous chapter demonstrated that a model can solve a mathematical problem in its original form but fail when the same problem is presented differently. This chapter asks whether robustness to perturbations can be taught during reinforcement learning with verifiable rewards (RLVR), and whether the value of this depends on how training problems are selected.

The motivation behind this comes from the fact that high accuracy on a standard mathematics benchmark does not necessarily imply reliable reasoning. GSM-Symbolic finds substantial variation when the numerical values in a problem template are changed (Mirzadeh et al., 2024), GSM-Plus shows that models can fail on previously solved problems after statements or question targets are altered (Li et al., 2024b), and ThinkBench reports weak performance on dynamically generated out-of-distribution variants (Huang et al., 2025b). A robust reasoner should respond to the mathematical relationships in a problem, not overfit to incidental wording or a familiar template.

Recent work provides empirical evidence that training on augmentations can improve generalisation. Transformation-Augmented GRPO reports gains on both competition mathematics and out-of-distribution reasoning benchmarks after training on equivalent rephrasings (Le et al., 2026). Question-begets-Question finds that curriculum-selected, teacher-generated variants transfer to harder unseen problems, while static augmentation plateaus (Bao et al., 2026). MuggleMath shows a limit to this effect: augmented training improves performance within the GSM8K and MATH problem families, but transfers weakly from augmented GSM8K to MATH and from augmented MATH to GSM8K (Li et al., 2024a). These results motivate testing augmentation while cautioning against assuming that more variants automatically produce broader robustness.

Curriculum learning has also been shown to improve generalisation in language model reasoning. AdaRFT reports higher final mathematical accuracy and fewer training steps when difficulty is adjusted using recent rewards (Shi et al., 2025). Self-Evolving Curriculum reports better generalisation to harder, out-of-distribution problems across mathematics, planning, and inductive reasoning (Chen et al., 2025). Frontier Learning evaluates on held-out generator levels that are unseen during training and reports stronger gains than fixed-pool baselines across several reasoning tasks (Ramesh et al., 2026). Together, these

studies suggest that reinforcement learning benefits from keeping training near the model’s changing competence boundary, while leaving open how such curricula interact with augmented problems.

Importantly, models are not equally vulnerable to every change. Section 3.4.2 (*Comparison between Augmentations*) of the previous chapter found that small lexical and nuisance edits were less damaging than richer changes to a problem’s structure. Subproblem augmentation, which embeds a required value behind an additional reasoning step, produced the most consistent failures across the evaluated models. This motivates a targeted intervention: rather than train on arbitrary perturbations, we post-train on the specific type of variation that exposed the clearest weakness.

This chapter brings these two ideas together. We introduce subproblem variants into reinforcement learning on a procedural Reasoning Gym task and compare fixed, randomized, and adaptive curricula. The central question is whether targeted augmentation improves robustness, and whether its effect depends on how the curriculum uses the resulting rewards. Our results show that augmentation is not uniformly beneficial: its value depends strongly on the curriculum and on whether augmented outcomes are allowed to control curriculum difficulty. The full analysis is presented in Section 4.4.

4.2 Scope and Contributions

This chapter builds on Reasoning Gym for procedural tasks and answer verification, and on Frontier Learning for curriculum-based selection of generator configurations (Stojanovski et al., 2025; Ramesh et al., 2026). The training loop is implemented in veRL (Sheng et al., 2025). Our contribution is not a new curriculum algorithm, but a study of how offline subproblem augmentation interacts with fixed, randomized, and adaptive curricula on the `decimal_arithmetic` task.

The contributions of this chapter are:

1. A hybrid subproblem-augmentation stage that combines exact programmatic construction with fixed, manually specified renderers and a reviewed bank of Qwen3-14B-generated wording templates, without loading a teacher model during RL without loading a teacher model during RL
2. A matched 200-step comparison of clean and augmented training under Uniform, PLR, DR, and Frontier Learning curricula
3. A curriculum-feedback intervention in which augmented prompts still train the policy, but only rewards from unaugmented prompts update Frontier Learning’s regret estimates and level-selection decisions
4. Common evaluation across clean in-domain ladders, external mathematics, and structured perturbation benchmarks, together with reproducible results

4.3 Experiment Setup and Methodology

4.3.1 Training environment and curricula

Procedural tasks and levels

Reasoning Gym

Reasoning Gym is an open-source library of procedural datasets for training and evaluating reasoning models (Stojanovski et al., 2025). Each task pairs a configurable question generator with a task-specific answer and scoring function. A seed and an item index deterministically produce an instance, allowing fresh questions to be generated procedurally. Because the answer is computed as part of generation, model responses can be checked automatically.

The library covers several forms of reasoning. It covers arithmetic, algebra, algorithmic, graph, logic, and planning tasks, including polynomial equations, shortest paths, Sudoku, and Sokoban. This chapter studies the Decimal Arithmetic task.

Decimal arithmetic

The decimal-arithmetic task generates expressions containing addition, subtraction, multiplication, and division. It samples decimal operands, operators, and parenthesis groupings, then evaluates the expression using decimal arithmetic. The configuration controls the range of decimal places per operand, the number of terms, and the arithmetic precision used to construct the reference answer. The accompanying scorer compares a submitted numerical answer with this generated reference under a precision-dependent tolerance.

In Frontier Learning, a *level* is a generator configuration rather than a single question. For decimal arithmetic, a level fixes three properties: the number of decimal places, the number of terms, and the arithmetic precision. Reasoning Gym can generate many questions at the same level by varying the numerical values, operators, and parentheses. The curriculum therefore selects the difficulty settings, and Reasoning Gym generates the individual training questions.

Figure 4.1 provides an overview of the complete training loop; the curriculum, optimisation, and augmentation components are defined in the sections that follow.

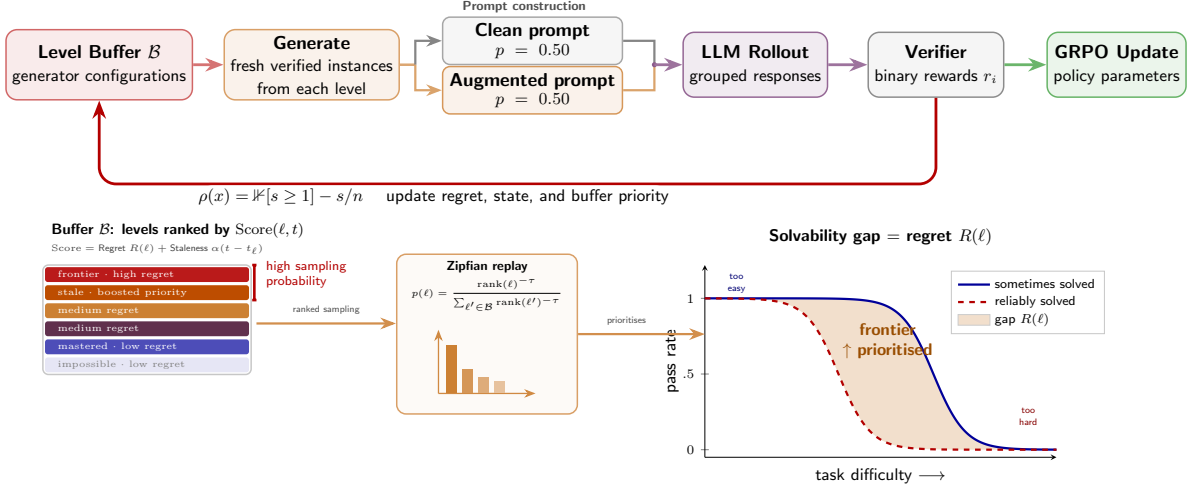


Figure 4.1: Frontier Learning and offline subproblem-augmentation pipeline. A sampled generator configuration produces fresh verified instances, which follow either the clean or augmented prompt path before grouped policy rollouts. Verifier rewards update the policy through GRPO and separately update curriculum regret. The mathematical construction and verifier targets are programmatic. Qwen3-14B is used offline only to propose alternative clue templates, which are reviewed before training. No teacher model is present in the RL loop.

Policy optimisation and regret

Training uses Group Relative Policy optimisation (GRPO), introduced by DeepSeekMath as a memory-efficient method for optimizing mathematical reasoning from groups of sampled responses (Shao et al., 2024). For each prompt, the policy samples a group of responses and the verifier marks each response as correct or incorrect. These grouped outcomes also provide a useful estimate of whether the prompt lies near the model’s current competence boundary. If every response is correct, the prompt is already mastered; if every response is incorrect, it may currently be too difficult. In both cases, the group contains little relative signal for a GRPO update. Prompts that are solved occasionally but not reliably are more informative: they show that success is within reach while leaving substantial room for improvement.

Frontier Learning turns this intuition into a score for each level in two stages (Ramesh et al., 2026). First, consider one problem instance x generated from a level. Let n be the number of responses sampled for that problem and let s_x be the number that are correct. The problem-wise rollout regret is

$$\rho(x) = \mathbb{I}(s_x \geq 1) - \frac{s_x}{n}. \quad (4.1)$$

Here, the first term records whether the problem was solved at least once, whereas s_x/n measures how reliably it was solved. Thus $\rho(x)$ is zero when all responses fail or all succeed, and large when success is possible but inconsistent. For example, one correct

response among eight gives $\rho(x) = 7/8$.

The curriculum assigns regret to a level ℓ as the expected problem-wise regret

$$R(\ell) = \mathbb{E}_{x \sim \ell}[\rho(x)]. \quad (4.2)$$

In practice, this expectation is estimated by averaging $\rho(x)$ over a rolling window of recent problem instances generated from that level. A high $R(\ell)$ therefore means that the level repeatedly produces problems the model can solve occasionally but not reliably. This is the level-wise quantity used to prioritize the curriculum buffer.

Frontier Learning curriculum

At each training step, Frontier Learning chooses several levels and Reasoning Gym generates problems at those levels. The model attempts each problem several times. The verified answers are used to update the model through GRPO and to update each level’s regret $R(\ell)$. Levels with higher regret are then more likely to be selected again.

The curriculum maintains a fixed-capacity buffer of level configurations, initialized from a small grid of seed levels. A training batch can be populated through three coupled mechanisms. *Replay* revisits buffered levels according to a priority that combines estimated regret with staleness, so useful levels are revisited without permanently excluding levels that have not been sampled recently. *Exploration* proposes configurations from outside the current buffer. *Mutation* makes local changes to the attributes of replayed levels, with the mutation rate conditioned on whether a level is unseen, impossible, productive, or mastered. Novel explored or mutated configurations can enter the buffer. Their subsequent rollout outcomes update their regret and state, which changes the distribution of levels selected in later steps. This closes the feedback loop: training changes policy competence, and measured competence changes the future training distribution.

Baselines, augmentation, and ablations

We compare the full Frontier Learning curriculum with two sampling baselines and one structural ablation:

- **Frontier Learning** combines prioritized replay with exploration and state-dependent local mutation in an evolving level buffer (Parker-Holder et al., 2022; Ramesh et al., 2026)
- **PLR** is the fixed-buffer ablation. It retains regret- and staleness-prioritized replay, but can select only from the eight initial seed levels (Jiang et al., 2021)
- **DR** is a diversity baseline. It independently samples fresh configurations without retaining a regret-ranked memory

- **Uniform** is the simplest baseline. It samples uniformly from the eight fixed seed levels, using neither prioritisation nor expansion

Each curriculum is evaluated under two matched prompt conditions:

- **Clean**: no subproblem augmentation
- **Augmented**: augmentation is sampled using the probabilities defined in the next section, while the remaining prompts stay clean

The paired runs use the same model, seed, training budget, and hyperparameters. This shows whether augmentation helps and whether its effect changes with the curriculum.

The augmentation branch shown in Figure 4.1 is detailed next. We additionally evaluate a curriculum-feedback ablation for augmented Frontier Learning, described below.

To test whether augmentation still benefits from adaptive problem selection, we also remove the curriculum entirely. This static-pool ablation contains 12,800 unique base problems, matching 200 steps of 64 prompts. The pool is balanced across PLR’s eight seed levels, with 1,600 problems from each level, and exactly half of the prompts at every level are augmented. The clean and augmented static-pool files contain the same underlying problems, answers, and order. Training therefore receives the same prompt budget as the curriculum runs, but problems are shuffled and consumed without prioritised replay or level adaptation.

4.3.2 Offline subproblem augmentation

Our augmentation stage modifies a generated decimal-arithmetic prompt before policy rollouts are sampled, while preserving its original numerical answer and verifier. The following sections describe the programmatic transformation, the sources and validation of its wording templates, and the probabilities governing its use during training.

Transformation and verification

Subproblem augmentation turns a direct decimal calculation into a composed reasoning problem. Instead of presenting every operand explicitly, the prompt represents one, or occasionally two, as variables defined by self-contained auxiliary problems. For example, rather than giving 2.5 directly in $2.5 + 3.7$, the prompt can state that x divided equally among five groups gives 0.5 per group, and then ask for $x + 3.7$. The model must infer $x = 2.5$, substitute it into the expression, and perform the original calculation. The arithmetic expression and final answer are therefore preserved, while the solution acquires an additional reasoning dependency. Each auxiliary problem is constructed backwards from the known operand, so its relationship is exact and the original answer verifier remains valid.

Variation comes from both the generated arithmetic problem and the transformation. The augments contains 15 mathematical families, organized into additive, multiplicative, geometric, sequence, and place-value groups. All 15 families are available during training. Each has three built-in phrasings, and six prompt layouts control how the clues and final expression are combined. An augmentation can also replace two operands or introduce an intermediate recovery step. Thus a family is a reusable construction rule rather than a fixed augmented question. Table 4.1 gives a compact overview; Appendix A.2.1 provides the complete family catalogue.

Table 4.1: The five groups of subproblem transformations, with representative families from each group.

Group	General construction	Representative families
Additive	Recover a quantity through a sum or difference.	Additive balance; subtraction remainder
Multiplicative	Recover a quantity through division, ratios, or rates.	Equal shares; unit price; percentage
Geometric	Recover a dimension from a geometric constraint.	Rectangle area; midpoint
Sequence/statistics	Infer a value from a sequence or summary statistic.	Missing average; arithmetic sequence
Place value	Recover a decimal through power-of-ten scaling.	Place value

Wording bank

The mathematical construction of each augmentation is programmatic. The code samples the auxiliary values, constructs the subproblem backwards from the replaced operand, and checks that solving it recovers that operand exactly. It also provides three fixed, manually specified phrasings for each of the 15 families, six layouts for combining the subproblems with the final expression, and three templates for two-step recovery.

Qwen3-14B was used offline only to propose alternative phrasings for the auxiliary problems. It did not generate numerical values, complete training problems, or verifier targets. The proposals were automatically checked for the required placeholders and then manually reviewed for mathematical correctness and clear variable binding. The approved bank contains 106 phrasings covering nine families. Families without an approved phrasing use the fixed templates. No teacher model is used during reinforcement learning.

Sampling probabilities

The augmentation settings are:

- **Augmentation probability 0.5:** half of the prompts are augmented and half retain their clean form;

- **Two-step probability 0.25:** an augmented prompt may require an additional intermediate step; and
- **Two-subproblem probability 0.15:** an augmented prompt may hide two operands rather than one.
- **Offline-wording probability 0.5:** when an augmented prompt uses one of the nine families covered by the approved bank, a Qwen3-14B-generated clue template is selected with probability 0.5; otherwise a built-in template is used

The latter three probabilities are conditional on a prompt being augmented.

Separating policy learning from curriculum feedback

Ordinary augmented Frontier Learning uses the verifier outcomes on augmented prompts for both the GRPO update and the regret-based level buffer. This confounds two sources of difficulty: failure may be caused by the procedural configuration or by the added presentation step. This motivates a specific hypothesis: augmented failures may distort the curriculum signal, causing the controller to prioritise or mutate levels because of presentation difficulty rather than the underlying generator difficulty. Augmented prompts could therefore remain useful for policy learning while being harmful when their rewards also control level selection. We test this hypothesis with a curriculum-feedback ablation. All clean and augmented prompts still contribute to the policy update, but only clean-prompt rewards update regret, level state, mutation, and admission decisions. With a 0.5 augmentation probability, curriculum feedback is consequently estimated from the clean half of the sampled prompts.

4.3.3 Training configuration

Training uses veRL, an open-source framework for distributed LLM post-training that orchestrates rollout generation and model updates across GPUs (Sheng et al., 2025). All primary runs initialize Qwen3-4B-Base and train for 200 GRPO steps with a batch of 64 prompts, eight rollouts per prompt, learning rate 10^{-6} , maximum prompt length 1,024, and maximum training response length 2,048. Validation is scheduled every 20 steps. The same binary decimal verifier and final step-200 checkpoint are used throughout.

Uniform and PLR draw from the same fixed pool of eight initial levels. Uniform samples them equally, whereas PLR preferentially revisits levels with high regret or those that have not been sampled recently. Frontier Learning begins with the same eight levels but can expand its training pool by mutating existing configurations. It selects four levels per step and can retain up to 100 levels. It allocates 30% of its selections to exploring new levels, while the remaining selections revisit levels retained in the buffer. The remaining controller hyperparameters are reported in Table A.8 in the reproducibility appendix.

4.3.4 Evaluation data

We separate evaluation according to what each dataset tests. *In-domain* evaluation uses clean held-out instances from the same `decimal_arithmetic` generator as training. *Out-of-domain transfer* is measured on GSM8K (Cobbe et al., 2021), MATH-500—a 500-problem subset of the MATH benchmark (Hendrycks et al., 2021; Hugging Face H4 Team, 2024)—and a 90-problem AIME collection distributed by AI-MO (AI-MO, 2024). *Perturbed robustness* is measured on benchmarks constructed by systematically modifying source problems: GSM-Symbolic (Mirzadeh et al., 2024), GSM-Plus (Li et al., 2024b), and the simple and hard MATH-Perturb sets (Huang et al., 2025a). We report this final group separately because it tests sensitivity to deliberate changes in a problem, rather than performance on a different clean benchmark alone.

For the in-domain evaluation, we construct held-out 10-level and 12-level validation sets. We call the 10-level validation set the *easy ladder* and the broader, more difficult 12-level set the *hard ladder*. Their exact configurations and relationship are defined in Appendix A.2.2.

Split	Levels	Examples	Decimal places	Terms	Required precision
Easy ladder	10	2,000	4–10	4–12	8–18 digits
Hard ladder	12	2,400	4–20	4–20	8–30 digits

Table 4.2: In-domain decimal-arithmetic validation ladders. The columns give the ranges across level configurations. The hard ladder covers a substantially broader and more difficult envelope.

All evaluations use greedy decoding with a 2,048-token response budget.

The external suite contains 3,067 examples: GSM8K (1,319), MATH-500 (500), AIME (90), three GSM-Symbolic subsets (250 total), seven GSM-Plus perturbations (350), and simple and hard mathematical perturbations (279 each).

The standard clean and augmented conditions for Uniform, PLR, DR, and Frontier Learning use training seeds 42 and 123. Tables report their mean and sample standard deviation. The clean-curriculum-scoring intervention and static-pool ablation use seed 42 and are labelled accordingly. The standard-condition learning curves average both training seeds and show their sample standard deviation.

4.4 Results and Analysis

4.4.1 Primary comparison: PLR and Frontier Learning

We first compare the two curriculum methods - PLR and Frontier Learning. Table 4.3 reports their final hard-ladder accuracy. Each entry is the mean and sample standard deviation across the two training seeds.

Curriculum	Clean training	Augmented training	Mean difference
PLR	18.48 ± 0.27	28.40 ± 3.39	$+9.92 \pm 3.12$
Frontier Learning	24.02 ± 8.28	28.23 ± 0.97	$+4.21 \pm 9.25$

Table 4.3: Accuracy (%) on the 2,400-example hard ladder with 2,048-token greedy decoding. Values are mean $\pm$ sample standard deviation across two training seeds. The final column summarises the paired within-seed changes.

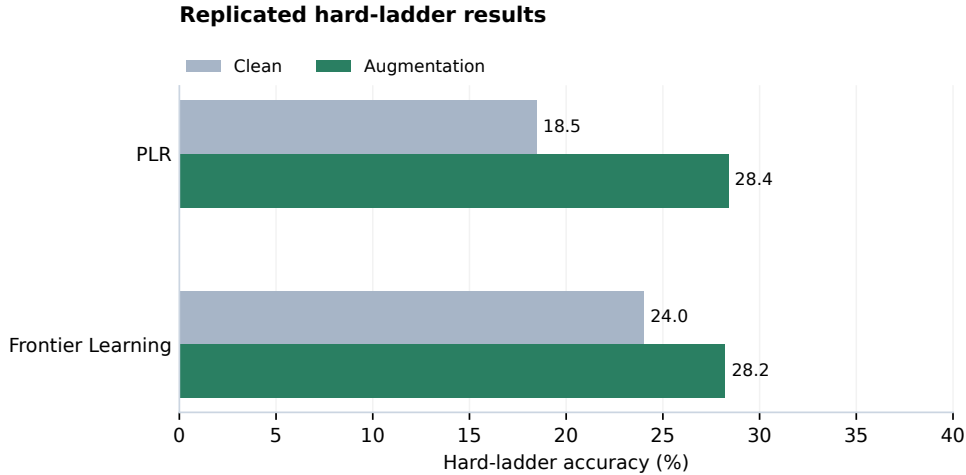


Figure 4.2: Clean and augmented hard-ladder accuracy under the two primary curricula. Bars show the two-seed mean; the corresponding sample standard deviations are reported in Table 4.3.

PLR shows the clearest result. Augmentation improves its mean accuracy by 9.92 percentage points, and both seeds improve individually: by 12.13 points for seed 42 and 7.71 points for seed 123. Frontier Learning has a positive mean difference, but its large deviation reflects a reversal across seeds: augmentation changes accuracy by -2.33 points for seed 42 and $+10.75$ points for seed 123. The in-domain evidence therefore supports a benefit for PLR with augmentations but not a stable direction for ordinary augmented Frontier Learning.

The level-wise results in Figure 4.3 show where these changes occur. PLR’s gain is spread across the easier and intermediate configurations, while the hardest levels remain close to zero. Frontier Learning varies substantially across seeds on the earlier levels and shows little gain at the difficult end of the ladder.

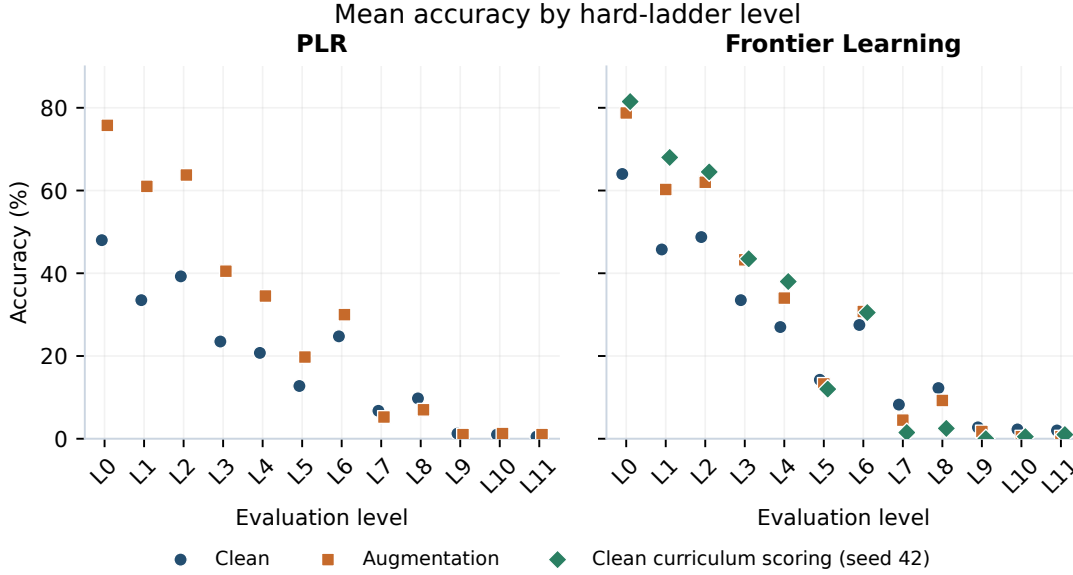


Figure 4.3: Mean accuracy by hard-ladder level for PLR and Frontier Learning at 2,048 tokens. Standard-condition markers show the two-seed mean. Across-seed variation over training is shown in Figure 4.6. The clean-curriculum-scoring markers are from seed 42 only and are analysed next.

The improvement is therefore not produced by averaging over one unusually favourable configuration. It mainly reflects better performance where the model can already solve at least some generated instances, rather than a breakthrough on the three most extreme levels.

4.4.2 Separating policy learning from curriculum feedback

The preceding result motivates the curriculum-feedback hypothesis introduced in the methodology: augmented failures may be useful training signals for the policy but misleading signals for the controller that chooses and mutates levels. We test that explanation by allowing both clean and augmented prompts to update the policy while allowing only clean-prompt rewards to update the Frontier Learning curriculum.

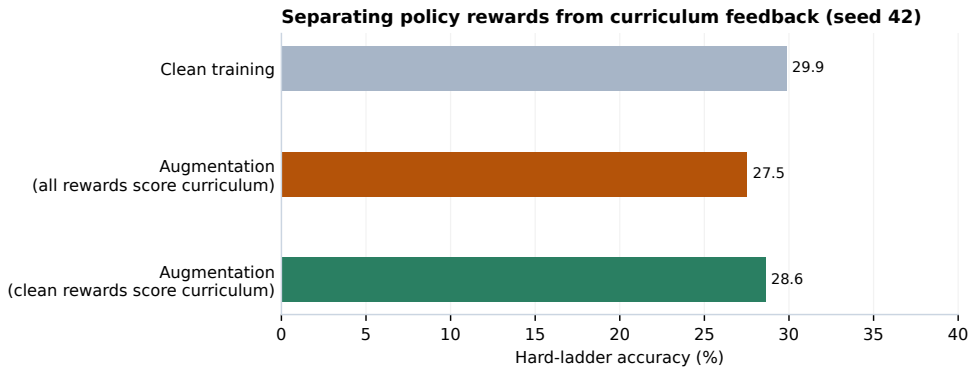


Figure 4.4: Frontier Learning with clean training, ordinary augmented training, and augmented policy updates with clean-only curriculum scoring.

Within seed 42, clean-only curriculum scoring raises augmented Frontier Learning from 27.54% to 28.63%, recovering part of the gap to clean training at 29.88%. On L0–L8 it obtains 38.00%, compared with 36.28% for ordinary augmented Frontier Learning and 38.56% for clean training. This recovery is consistent with the hypothesis that some of the harm in this seed arises when presentation-related failures alter level prioritisation and mutation.

4.4.3 Uniform and domain-randomisation baselines

Sampling strategy	Clean training	Augmented training	Mean difference
Uniform	22.69 ± 6.51	27.31 ± 1.86	$+4.63 \pm 4.66$
DR	23.85 ± 8.04	23.15 ± 5.63	-0.71 ± 2.42

Table 4.4: Baseline accuracy (%) on the hard ladder with 2,048-token greedy decoding. Values are mean $\pm$ sample standard deviation across two training seeds. The final column summarises the paired within-seed changes.

Uniform sampling and domain randomisation (DR) provide contextual baselines. Unlike PLR, Uniform does not prioritise difficult levels; unlike Frontier Learning, neither baseline uses the full regret-driven level-expansion mechanism.

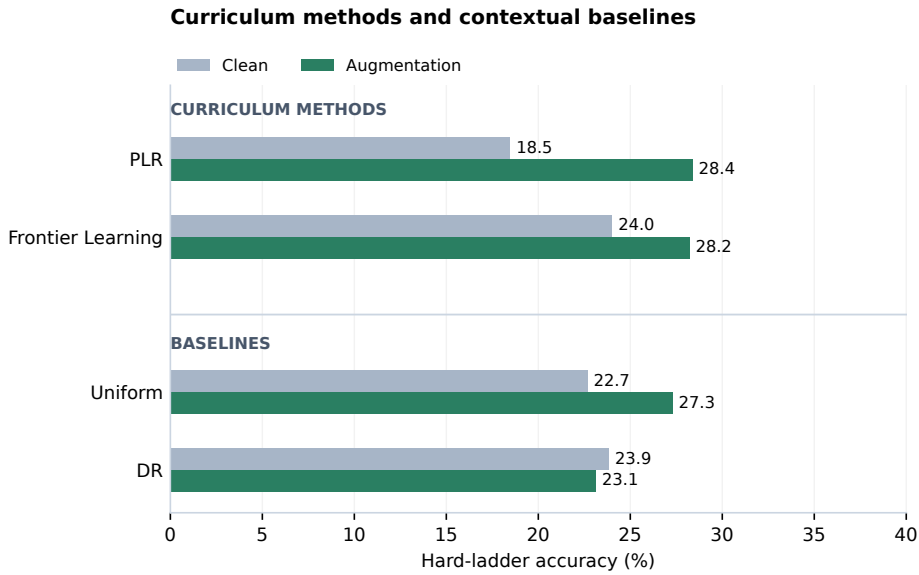


Figure 4.5: Clean and augmented results for all four sampling strategies at 2,048 tokens. Bars show the two-seed mean, while Tables 4.3 and 4.4 report the sample standard deviations. PLR and Frontier Learning are curriculum methods; Uniform and DR are contextual baselines.

Uniform improves under augmentation in both seeds, although its mean gain of 4.63 points is less than half PLR’s. DR changes by -2.42 and $+1.00$ points across the two seeds,

producing a near-zero mean. Augmentation can therefore help without prioritisation, but PLR provides the largest and most consistent in-domain gain.

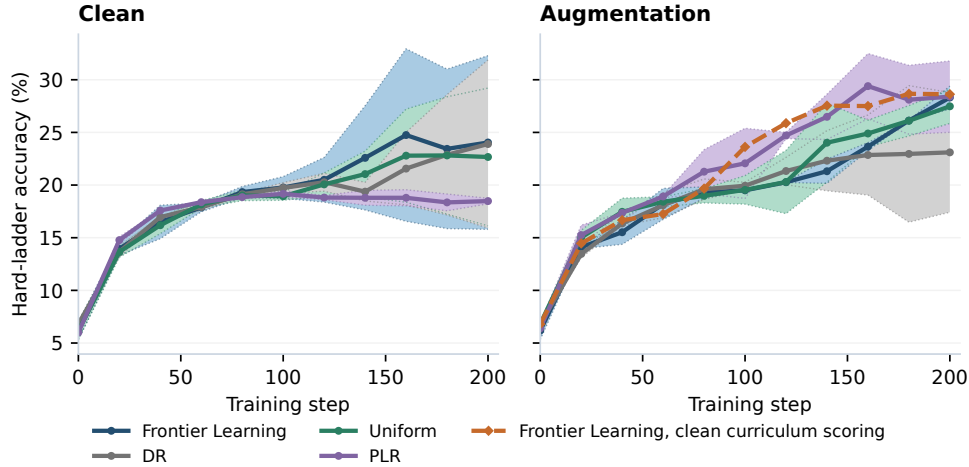


Figure 4.6: Hard-ladder validation accuracy measured every 20 steps. Solid lines show two-seed means and shaded regions show one sample standard deviation. The dashed line is the single-seed clean-curriculum-scoring intervention.

4.4.4 Easy-ladder comparison of curriculum methods

The hard ladder includes configurations that remain close to unsolved after training. We repeat the PLR and Frontier Learning comparison on the narrower easy ladder to test augmentation over a more solvable difficulty range. Table 4.5 reports the two-seed results, while Figure 4.7 shows the individual runs.

Curriculum	Clean	Augmentation	Mean difference
PLR	31.08 ± 0.88	52.30 ± 4.74	$+21.23 \pm 3.85$
Frontier Learning	42.43 ± 16.30	52.93 ± 1.80	$+10.50 \pm 18.10$

Table 4.5: Accuracy (%) on the 2,000-example easy ladder with 2,048-token decoding. Values are mean $\pm$ sample standard deviation across two seeds. The final column summarises paired within-seed changes.

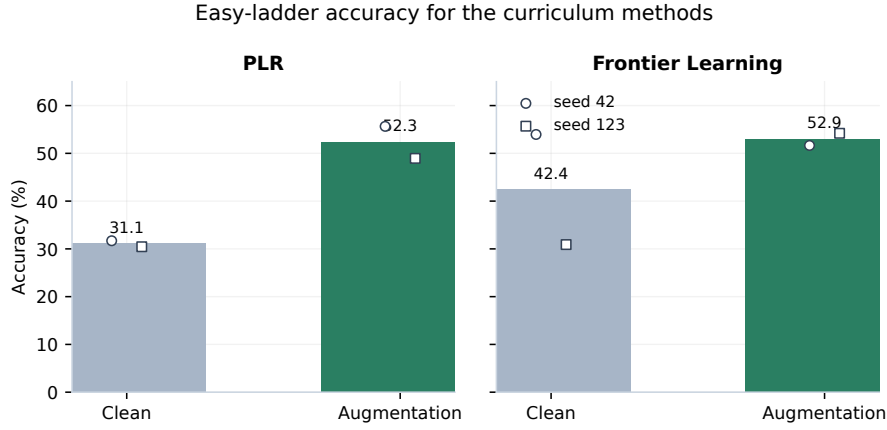


Figure 4.7: Easy-ladder accuracy for PLR and Frontier Learning at a 2,048-token decoding budget. Bars show the two-seed mean and markers show the individual runs; sample standard deviations are reported in Table 4.5.

PLR improves in both runs: by 23.95 points for seed 42 and 18.50 points for seed 123. Its larger gain on the easy ladder supports the level-wise finding that augmentation is most useful on configurations already within the model’s solvable range. Frontier Learning has a positive mean difference, but not a consistent paired effect: augmentation changes accuracy by -2.30 points for seed 42 and $+23.30$ points for seed 123. Its augmented results are nevertheless much closer across seeds, at 51.65% and 54.20%, than its clean results, at 53.95% and 30.90%. The easy-ladder results therefore strengthen the PLR finding. For Frontier Learning, however, augmentation improves one seed and slightly reduces performance in the other.

4.4.5 External and perturbed evaluation

We separate clean transfer from robustness to perturbations because they answer different questions. GSM8K, MATH-500, and AIME test whether decimal-arithmetic post-training transfers to established mathematical reasoning benchmarks.

Run	GSM8K	MATH-500	AIME
PLR clean	91.75 $\pm$ 0.07	74.50 $\pm$ 3.25	10.00 $\pm$ 1.56
PLR + augmentation	92.35 $\pm$ 1.06	77.40 $\pm$ 0.57	9.45 $\pm$ 0.78
Frontier Learning clean	91.00 $\pm$ 1.41	75.10 $\pm$ 0.14	10.00 $\pm$ 0.00
Frontier Learning + augmentation	90.25 $\pm$ 1.48	74.30 $\pm$ 0.99	8.90 $\pm$ 0.00
Frontier Learning + augmentation, clean curriculum scoring [†]	92.00	77.20	15.60
Uniform clean	92.30 $\pm$ 0.42	73.60 $\pm$ 2.55	9.45 $\pm$ 0.78
Uniform + augmentation	91.95 $\pm$ 0.49	74.50 $\pm$ 1.84	10.55 $\pm$ 3.89
DR clean	92.15 $\pm$ 0.49	72.80 $\pm$ 0.28	8.90 $\pm$ 0.00
DR + augmentation	91.65 $\pm$ 0.07	75.50 $\pm$ 2.12	13.85 $\pm$ 0.78

Table 4.6: Accuracy (%) on clean external benchmarks with 2,048-token decoding. Standard conditions report mean $\pm$ sample standard deviation across two seeds. [†]The clean-curriculum-scoring intervention uses seed 42 only. AIME contains 90 examples, so its differences are especially uncertain.

Across seeds, PLR augmentation improves MATH-500 by 2.90 points and GSM8K by 0.60 points. Ordinary augmented Frontier Learning declines on all three means, and each decline has the same direction in both seeds. The clean-curriculum-scoring checkpoint exceeds its seed-42 clean counterpart on all three benchmarks, but this intervention has not been repeated.

The second group measures robustness to altered symbols, wording, numerical details, and mathematical presentation. These datasets are reported separately from the clean benchmarks.

Run	GSM-Symbolic	GSM-Plus	Perturb.-simple	Perturb.-hard
PLR clean	79.00 $\pm$ 4.24	84.71 $\pm$ 0.61	62.70 $\pm$ 0.99	38.40 $\pm$ 0.00
PLR + augmentation	84.50 $\pm$ 0.71	84.29 $\pm$ 0.81	66.15 $\pm$ 1.77	42.45 $\pm$ 0.78
Frontier Learning clean	80.83 $\pm$ 2.59	83.86 $\pm$ 0.61	60.55 $\pm$ 0.49	41.40 $\pm$ 1.84
Frontier Learning + augmentation	76.50 $\pm$ 3.54	83.29 $\pm$ 0.61	61.15 $\pm$ 1.77	38.70 $\pm$ 0.99
Frontier Learning + augmentation, clean curriculum scoring [†]	83.33	85.71	65.90	41.60
Uniform clean	81.83 $\pm$ 0.24	84.71 $\pm$ 1.41	61.65 $\pm$ 3.61	38.85 $\pm$ 1.77
Uniform + augmentation	78.83 $\pm$ 1.18	85.71 $\pm$ 2.02	63.05 $\pm$ 3.04	40.65 $\pm$ 1.77
DR clean	83.00 $\pm$ 4.24	84.43 $\pm$ 1.01	62.90 $\pm$ 1.84	40.85 $\pm$ 2.05
DR + augmentation	85.00 $\pm$ 1.41	83.57 $\pm$ 0.20	62.55 $\pm$ 0.21	41.40 $\pm$ 1.27

Table 4.7: Accuracy (%) on perturbed benchmarks with 2,048-token decoding. Standard conditions report mean $\pm$ sample standard deviation across two seeds. [†]The clean-curriculum-scoring intervention uses seed 42 only. GSM-Symbolic averages three subsets and GSM-Plus averages seven perturbation types.

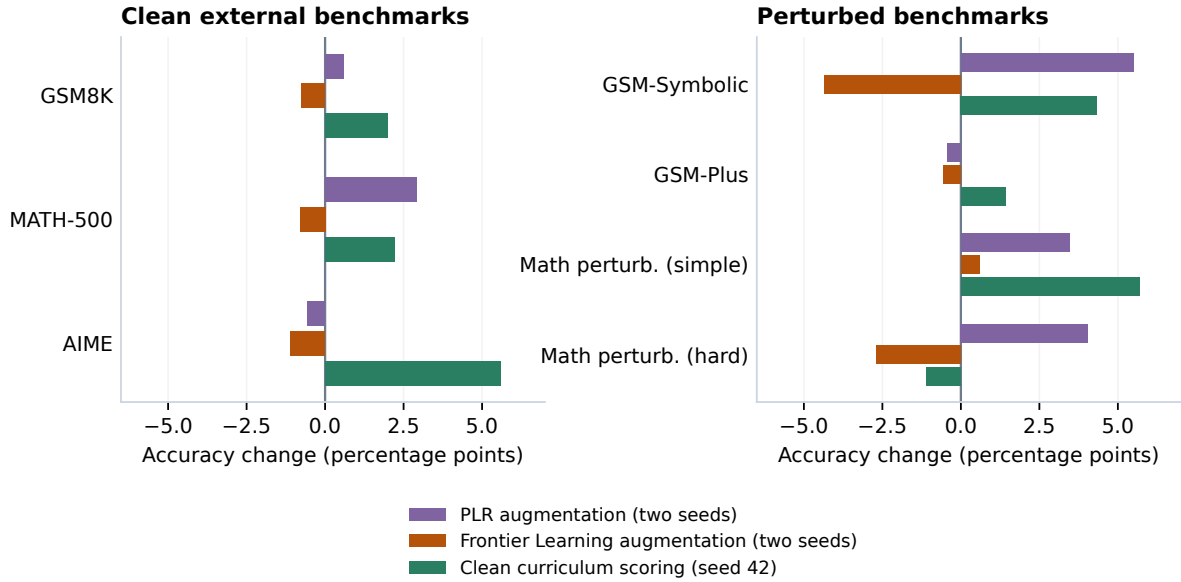


Figure 4.8: Change relative to clean training for the primary augmentation comparisons. For the standard PLR and Frontier Learning conditions, bars show the mean paired change across two seeds. Tables 4.6 and 4.7 report the absolute accuracies and sample standard deviations. Clean curriculum scoring is a seed-42 comparison.

The perturbation results provide the clearest replication. PLR augmentation improves GSM-Symbolic by 5.50 points, the simple mathematical perturbations by 3.45 points, and the hard perturbations by 4.05 points on average; all three improvements have the same direction in both seeds. GSM-Plus changes only slightly. Ordinary augmented Frontier Learning declines on GSM-Symbolic, GSM-Plus, and the hard perturbation set in both seeds, while its simple-set change is small and inconsistent. Uniform and DR remain mixed across datasets and seeds. The seed-42 clean-curriculum-scoring intervention recovers much of the ordinary Frontier Learning loss.

4.4.6 Interpretation

Why PLR benefits. PLR seeds the level buffer with eight generator configurations. Without augmentation, its mathematical and presentation diversity are both constrained. Augmentation creates many exact instances and surface forms inside those fixed levels, apparently reducing overfitting to their canonical presentation. Consistent gains are demonstrated across seeds and both evaluation ladders, as well as external OOD benchmarks.

Why ordinary Frontier Learning does not. Frontier Learning already expands mathematical diversity through exploration and mutation. An augmented failure can cause an otherwise suitable configuration to be classified as too difficult, changing replay and mutation decisions. Ordinary augmentation therefore perturbs both the learner and the controller. With clean curriculum scoring, augmented prompts still train the

policy, but only clean rewards guide the curriculum. In the seed tested, separating these roles improves performance relative to ordinary augmented Frontier Learning, supporting the curriculum-feedback hypothesis without establishing its generality. A retrospective controller audit provides additional, though descriptive, support. Over the final 50 steps, clean curriculum scoring maintains higher mean buffer regret (0.229 versus 0.180) and higher mean training-batch return (0.395 versus 0.324), while overall buffer turnover and sampling concentration remain similar. The sampled configuration trajectories also diverge completely by the end of training. Full traces and summary statistics are reported in Appendix A.2.2.

4.5 Ablation study: removing the curriculum

We next compare three augmented runs at the common 2,048-token validation budget: the static pool, PLR, and Frontier Learning with clean-only curriculum scoring. This ablates adaptive selection while holding the model, number of training steps, batch size, seed, and augmentation rate fixed.

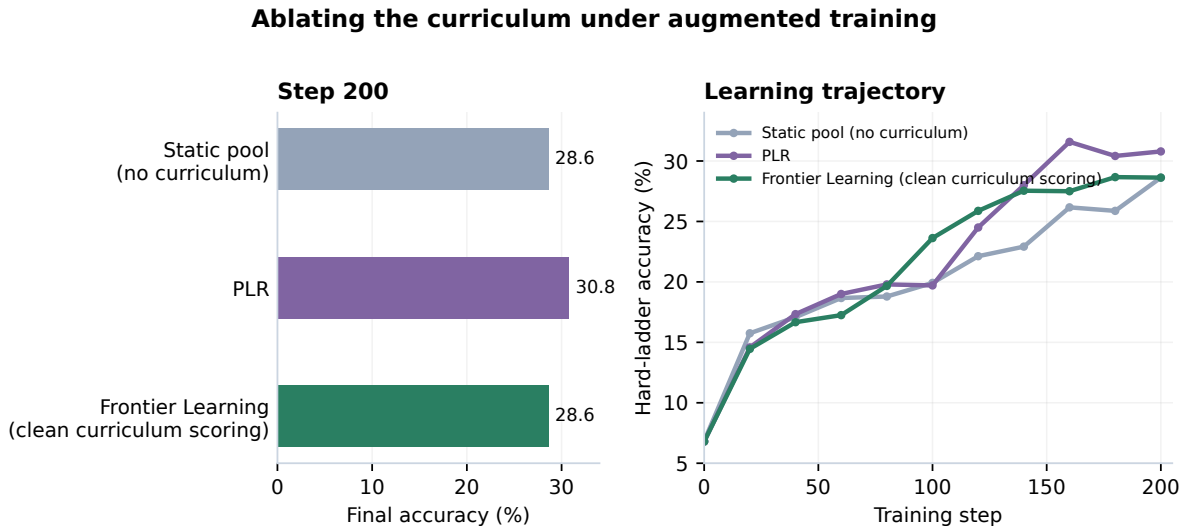


Figure 4.9: Curriculum ablation under augmented training. The left panel compares final step-200 hard-ladder accuracy, the right panel shows validation accuracy throughout training.

Figure 4.9 shows that the static pool is already a strong control: it reaches 28.63%, equal to Frontier Learning with clean-only scoring and 2.17 points below PLR’s 30.79%. The trajectories nevertheless show an advantage from adaptive selection before the end of training. Averaged over the common validation points through step 180, accuracy is 19.75% for the static pool, 21.46% for PLR, and 21.15% for Frontier Learning. PLR reaches 28.00% by step 140, whereas the static run does not reach comparable accuracy until step 200. Both curriculum methods improve training efficiency in this comparison. PLR

also achieves a higher final in-domain accuracy than static-pool training, while Frontier Learning with clean-only curriculum scoring matches the static pool’s final accuracy.

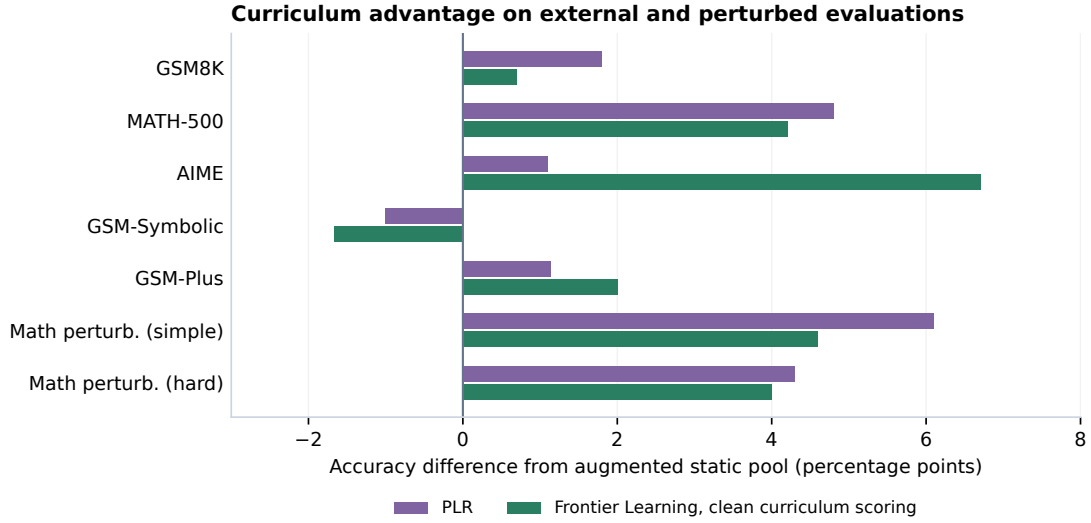


Figure 4.10: Transfer advantage of the two augmented curriculum runs over augmented static-pool training. Bars report absolute accuracy differences in percentage points; positive values favour the curriculum run.

The transfer results give a clearer separation. Both curriculum variants outperform the static model on six of the seven external and perturbed aggregates in Figure 4.10. Relative to static training, PLR gains 4.8 points on MATH-500, 6.1 on the simple mathematical perturbations, and 4.3 on the hard perturbations. Frontier Learning gains 4.2, 4.6, and 4.0 points on those same sets, respectively, and has the largest AIME gain at 6.7 points. Both are slightly worse on GSM-Symbolic. Overall, the ablation suggests that static augmented training accounts for much of the final in-domain improvement, while curriculum selection adds efficiency gains and broader transfer.

4.6 Limitations

This study focuses on decimal arithmetic within Reasoning Gym. Exact generation and verification make it a useful setting for isolating curriculum–augmentation interactions, but the resulting behaviour may not transfer directly to other procedural tasks or to domains with less reliable verifiers.

The training and evaluation budgets were also bounded by the available compute. Each policy was trained for 200 GRPO steps, so the experiments compare models at a fixed budget rather than after demonstrated convergence.

The model-generated component of the augmenter is limited to the reviewed Qwen3-14B clue bank. This bank covers nine of the 15 construction families. The rest are implemented programmatically. The resulting augmentation is reproducible and does not require a teacher during RL, but its linguistic diversity is bounded by a finite template bank. This

is because placing a sufficiently capable teacher inside the RL loop would add substantial inference latency and memory demand to every training step. Our infrastructure also did not provide access to external model APIs. The available local teachers were less suitable for unrestricted online generation: preliminary outputs frequently contained invalid or answer-changing rewrites. The verified offline bank therefore offers a practical compromise, although it provides less open-ended linguistic variation than a stronger teacher might produce. Future work could generate candidates asynchronously with a larger model and retain automatic answer verification before they enter training.

The standard comparisons use only two training seeds, while the clean-curriculum-scoring intervention and static-pool ablation use one. The reported standard deviations therefore describe the observed runs but provide limited information about variability across training seeds. The feedback intervention and controller audit support the proposed explanation, but further seeds are needed to establish whether the observed recovery is reproducible.

Finally, the post-training experiments consider only subproblem augmentation. This augmentation was selected because Chapter 3 identified it as a consistent source of failure across the evaluated models. Holding the augmentation type fixed enables a detailed study of its interaction with different task-selection strategies, but the resulting conclusions may not extend to other transformations. Future work should repeat these comparisons with augmentation families that alter wording, numerical details, and problem structure in different ways.

4.7 Conclusion

Post-training language models on targeted augmentations can substantially improve performance. At the same 200-step budget, it raises PLR hard-ladder accuracy from $18.48 \pm 0.27\%$ to $28.40 \pm 3.39\%$ across two seeds. Both runs improve individually. The gain extends beyond the training distribution: augmented PLR improves MATH-500, GSM-Symbolic, and both mathematical perturbation sets in both seeds. The offline design achieves this without placing a teacher model in the RL loop, making additional presentation diversity practical under a limited inference budget.

The results also suggest how augmentation can be used with a changing learning frontier. When augmented prompts train the policy but only clean rewards guide Frontier Learning, hard-ladder accuracy rises from 27.54% to 28.63%, partially closing the gap to the 29.88% clean checkpoint. This suggests that augmented experience can help the policy without necessarily providing a reliable signal for curriculum selection.

These benefits are not automatic. The in-domain effect reverses across the two ordinary Frontier Learning seeds, and its external performance generally declines under augmentation while Uniform and DR change little. Augmentation must therefore be applied intelligently at the frontier: augmented experience can strengthen the policy, but

it should not be allowed to distort the signal that selects the next training problems.

Decimal arithmetic makes this interaction visible because its problems and rewards are exact and short-horizon. The next chapter asks whether curriculum learning remains useful in interactive terminal tasks, where trajectories are longer and informative training experience is harder to identify.

Chapter 5

Curriculum Learning for Terminal-Agent Post-Training

5.1 Background and Motivation

The preceding chapters studied the effects of controlling the presentation of data to language models during reinforcement-learning post-training in two ways:

- *Augmentation*, which controls *how* existing data can be modified, and
- *Selection*, which controls *which* data point to present next

This chapter carries the second question into terminal-agent post-training.

A terminal agent is a language-model policy that completes tasks by iteratively writing code, invoking tools and commands, and acting on feedback from an executable environment. Terminal agents like Claude Code (Anthropic, 2025) and OpenAI Codex (OpenAI, 2025) have gained enormous popularity for their ability to solve tasks in many domains such as Software Engineering and Cybersecurity, and have demonstrated strong performance on long-horizon terminal benchmarks like TerminalBench (Merrill et al., 2026) and SWE-bench (Jimenez et al., 2024).

The success of these proprietary systems has encouraged the development of open-source models and training recipes. TerminalTraj constructs verified trajectories in containerized software environments (Wu et al., 2026), CLI-Gym and TermiGen generate executable tasks at scale (Lin et al., 2026; Zhu et al., 2026), and Nemotron-Terminal and OpenThoughts-Agent study synthetic data, filtering, and mixtures of agentic domains (Pi et al., 2026; Raoof et al., 2026). Reinforcement-learning systems including Endless Terminals, TMax, ROME, and SWE-Master further show that execution feedback can improve terminal and software-engineering agents (Gandhi et al., 2026; Ivison et al., 2026; Wang et al., 2025; Song et al., 2026).

Terminal-agent reinforcement learning combines expensive interactions with sparse and uneven feedback. A rollout may span dozens of model–environment turns, tens of thousands of generated tokens, and several minutes of sandbox execution (Wang and Ammanabrolu, 2025; Yuan et al., 2026). With group-relative advantages, a task provides policy-gradient signal only when its rollout group contains different outcomes. All-failure groups are common for small models on difficult tasks, while all-success groups indicate tasks that may already be mastered. In both cases, the interaction produces zero relative advantage.

One way to address this sparse feedback problem is to choose training tasks using the outcomes of earlier rollouts. This may direct more of the fixed interaction budget towards tasks that can produce a useful update. Task selection is also relevant to robustness. (Yao et al., 2026) show that the quality of training environments and verifiers affects generalisation in agentic reinforcement learning. We hold the verifiers fixed, but the curriculum changes which executable environments contribute training trajectories. Curriculum learning therefore provides a way to control environment exposure during post-training.

This chapter addresses these questions by developing a curriculum-learning pipeline for terminal tasks. Tasks are grouped into interpretable regions, rollout outcomes are used to estimate where the policy is learning, and future interactions are directed towards those regions. Concretely, the chapter asks two research questions:

1. Can grouped verifier outcomes identify a useful learning frontier for terminal-agent post-training?
2. Compared with uniform sampling, does prioritizing this frontier improve task completion both in domain and out of distribution?

5.2 Scope and Contributions

TMax provides the task-generation recipe, taxonomy, environments, verifiers, and the source training corpus used in this study (Iverson et al., 2026). The reinforcement-learning implementation builds on Open-Instruct and its asynchronous Reinforcement Learning stack (Allen Institute for AI, 2026) using Divergence Proximal Policy optimisation (DPPO) (Qi et al., 2026). The use of regret to follow a moving frontier is informed by PLR, ACCEL, and Frontier Learning work (Jiang et al., 2021; Parker-Holder et al., 2022; Ramesh et al., 2026).

Contribution of this thesis. The contribution is the design, implementation, and evaluation of curriculum learning for multi-turn reinforcement learning in heterogeneous executable environments. Concretely, it comprises:

1. A level representation, coverage phase, and regret-based curriculum built from canonical TMax metadata
2. A hierarchical replay algorithm evolved from Frontier Learning that first prioritizes task regions by regret and staleness, then samples distinct task instances within each selected region, while accounting for delayed asynchronous rollout results
3. A controlled evaluation that measures in-domain performance and OOD generalisation of curriculum methods and baselines.

5.3 Experimental Setup

5.3.1 Dataset

We train on the TMax-15K corpus introduced by (Iverson et al., 2026). The version of the published training split used in this study contains 14,601 unique task identifiers. Each task contains a natural-language instruction, an executable environment, and programmatic tests. Our execution backend requires each task environment to be materialized as an Apptainer overlay. After build retries, this succeeded for 13,795 tasks. We excluded the remaining 806 because their environments could not be built in the cluster setup: 289 failed during Python-package installation, 145 required unsuccessful external downloads, 70 had Python binary-compatibility conflicts, 300 failed during other Apptainer build steps, one relied on an invalid setup assumption, and one failed during overlay extraction. This is an infrastructure-compatibility filter rather than a selection based on task outcomes.

After holding out validation examples, our training split contains 13,555 verifiable terminal tasks and the validation split contains 240 tasks. Each task provides a natural-language instruction, an isolated executable environment, and a programmatic verifier. The corpus also records descriptive metadata such as skill type, primitive skills, scenario, and implementation language. For curriculum sampling, we use the following three categorical axes:

- **Domain** (9 values): data processing, data querying, data science, debugging, file operations, scientific computing, security, software engineering, and system administration.
- **Task complexity** (4 values): short, moderate, complex, and intricate.
- **Command complexity** (3 values): bash-only, bash and code, and bash, code, and system services.

All $9 \times 4 \times 3 = 108$ combinations occur in the training data.

5.3.2 Terminal tasks and rewards

When attempting the task, the policy π_θ interacts with the environment over a sequence of steps. At step t , it receives an observation o_t and produces an action a_t . The resulting trajectory is therefore

$$\tau = (o_0, a_0, o_1, a_1, \dots, o_T, a_T), \quad (5.1)$$

where o_0 is the initial observation and each subsequent observation reflects the state of the environment after the preceding action, where actions include text and tool calls and

observations include their outputs. A rollout ends when the policy submits, reaches the interaction limit, or encounters an environment timeout. Task-specific tests then assign

$$R(x, \tau) \in \{0, 1\}, \quad R(x, \tau) = 1 \iff \tau \text{ passes the verifier.} \quad (5.2)$$

5.3.3 From Tasks to Levels

Following the terminology of Prioritized Level Replay (PLR), a *level* is the unit whose sampling priority is tracked. We define a level as one of the 108 axis combinations

$$\ell(x) = (\text{domain}(x), \text{task_complexity}(x), \text{command_complexity}(x)). \quad (5.3)$$

A level is therefore a bucket of tasks with the same domain, task complexity, and command complexity. The three axes come from the TMax generation framework. TMax uses domain to organise task coverage and defines task complexity and command complexity as explicit difficulty controls. We reanalyse the solution artifacts released with TMax to check whether these categories correspond to measurable differences in solvability. The release contains eight Gemini-3-Flash-Preview rollouts for each of 8,843 tasks from the original corpus. As Figure 5.1 shows, mean pass@1 falls from 59.2% on short tasks to 51.9% on complex tasks, and from 57.8% for Bash-only tasks to 54.5% for tasks involving code and system services. Success also varies from 29.4% to 66.7% across domains.

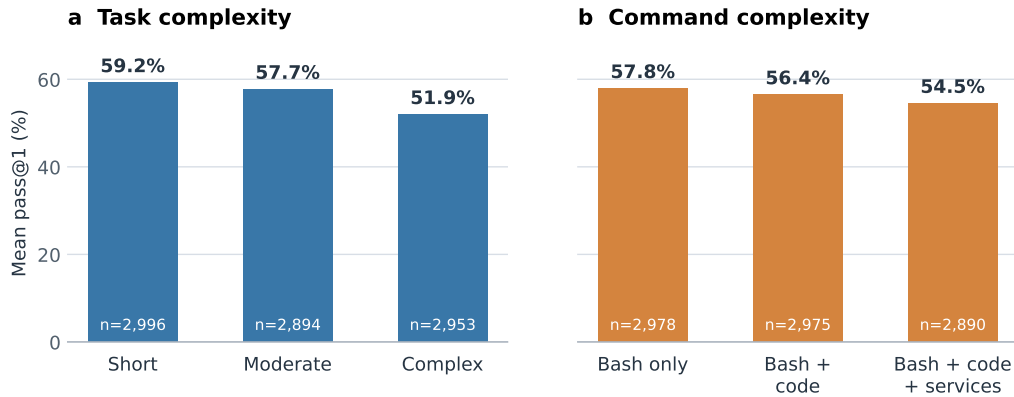


Figure 5.1: Mean pass@1 across the native TMax task- and command-complexity axes. Each task contributes equally and has eight Gemini-3-Flash-Preview rollouts. Task metadata and solution traces were released by (Iverson et al., 2026). The aggregation and this visualisation are ours. The 8,843-task artifact covers the three original task-complexity categories.

5.3.4 Model and execution environment

The policy is Qwen3.5-2B (Qwen Team, 2026b). Training uses the Open-Instruct reinforcement-learning stack (Allen Institute for AI, 2026). The agent acts by generating text, code, and tool calls. Tool calls are executed inside isolated Apptainer

environments (Kurtzer et al., 2017), and the task tests are run only when the trajectory is verified. No learned reward model is used.

5.3.5 Experimental conditions

We compare three ways of selecting training tasks:

- **Task-uniform (DR)** follows a shuffled ordering of all training tasks.
- **Level-uniform** first chooses metadata groups uniformly and then chooses tasks within those groups.
- **PLR** gives higher probability to metadata groups on which success is possible but inconsistent.

All three conditions begin from the same intermediate policy checkpoint. This checkpoint is produced by 54 initial training steps that visit every metadata group once. Thus the comparison starts from the same model and the same initial task exposure. The construction of this checkpoint and the three sampling rules are defined fully in Section 5.4.

5.3.6 Training configuration

Every selected task is attempted 16 times independently. A training batch contains eight tasks, and sampling uses temperature 1.

Setting	Value
Policy	Qwen3.5-2B
Tasks per batch	8
Trajectories per task	16
Sampling temperature	1
Prompt-token limit	2,048
Response-token limit	65,536
Maximum tool-interaction steps	48

Table 5.1: Main training settings shared by all experimental conditions.

The complete optimisation, curriculum, and execution configuration is provided in Appendix A.3.1.

The system uses three processes to generate and execute task attempts and one process to update the model. The learner is implemented with DeepSpeed ZeRO Stage 3 (Rajbhandari et al., 2020), with optimizer offload enabled and parameter offload disabled. This compute layout is fixed across all conditions. The logical roles of the learner and rollout processes, and the handling of stale trajectories, are defined in Section 5.4.

Comparisons are made after the same number of task-sampling steps. We separately record the number of model updates, attempted trajectories, retained task groups, and generated tokens.

5.3.7 Evaluation protocol

In-distribution evaluation. Final evaluation uses all 240 tasks in the held-out validation split. For each task, we sample one trajectory at temperature 1 and report pass@1. We also use a smaller evaluation set of 21 easy bash-only tasks to diagnose replay buffer statistics and prioritisation signal. Every checkpoint uses the same task identifiers, system prompt, tool parser, sandbox images, verifier versions, token limits, and interaction limit.

We report overall task success together with breakdowns by domain, task complexity, and command complexity. Environment-reset failures, timeouts, truncated trajectories, non-submitting trajectories, and empty responses are recorded separately so that infrastructure failures are not mistaken for model failures.

Out-of-distribution evaluation. We evaluate the final checkpoints on InterCode Bash (Yang et al., 2023). Its 200-task test split measures whether an agent can solve unseen interactive shell tasks using execution feedback. We use the same decoding configuration for every checkpoint and report pass@1.

We considered Terminal-Bench 2.1 (Merrill et al., 2026) and OpenThoughts-TBLite (OpenThoughts-Agent Team et al., 2026), but could not include them within the available experimental infrastructure. Their supported evaluation workflow uses the Harbor harness and Docker or Podman-based container execution, whereas the cluster used for this study provides none of Harbor, Docker, or Podman. Adapting and validating that execution stack was outside the remaining experimental budget. We therefore use InterCode-Bash as the external evaluation and treat the absence of evaluation on these broader benchmark suites as a limitation.

5.4 Curriculum-Learning Algorithm

The algorithm adapts the rollout-regret signal from Frontier Learning (Ramesh et al., 2026) to a corpus of executable tasks. It has four components: tasks are organized into levels, rollout outcomes estimate the learning value of each level, PLR selects future levels, and asynchronous RL updates the policy.

The curriculum operates on the 108 metadata groups introduced in Section 5.3.3. For one visit to task x , the policy generates G trajectories. If $R_{x,g} \in \{0, 1\}$ is the verifier reward of trajectory g , we compute

$$\bar{R}_x = \frac{1}{G} \sum_{g=1}^G R_{x,g}, \quad (5.4)$$

$$r(x) = \max\left(\mathbb{I}\left[\max_g R_{x,g} > \frac{1}{2}\right] - \bar{R}_x, 0\right). \quad (5.5)$$

The regret is zero when all trajectories fail or all succeed. It is positive when the task is solved at least once but not reliably. For example, one success among 16 trajectories gives $r(x) = 15/16$.

Each level ℓ stores a FIFO window $H_k(\ell)$ containing the regrets from its 16 most recent task visits. Its empirical regret is

$$\hat{r}_k(\ell) = \begin{cases} \frac{1}{|H_k(\ell)|} \sum_{u \in H_k(\ell)} u, & |H_k(\ell)| > 0, \\ 0, & |H_k(\ell)| = 0. \end{cases} \quad (5.6)$$

5.4.1 Prioritized level selection

PLR combines empirical regret with staleness (Jiang et al., 2021). Let τ_ℓ be the sampling step at which level ℓ was last drawn. With L levels and K level draws per batch, one replay horizon is $H_{\text{replay}} = L/K$. The priority score is

$$s_k(\ell) = \hat{r}_k(\ell) + \lambda_{\text{stale}} \min\left(\frac{\max(k - \tau_\ell, 0)}{H_{\text{replay}}}, 1\right). \quad (5.7)$$

Normalizing staleness by a replay horizon prevents the age term from overwhelming the regret signal, which is bounded by one. Levels are ranked by decreasing score and sampled without replacement according to

$$P_k(\ell) \propto \text{rank}_k(\ell)^{-\alpha}. \quad (5.8)$$

We use $L = 108$, $K = 2$, $\lambda_{\text{stale}} = 0.3$, and $\alpha = 2$. Thus one replay horizon is $H_{\text{replay}} = 54$ sampling steps.

Tied levels receive equal probability. If every score is equal, sampling is uniform. The last-draw time is updated when a level is selected, rather than when its delayed rollouts return.

After selecting K distinct levels, the sampler draws m distinct tasks uniformly from each level. We use $m = 4$, so one batch contains $B = Km = 8$ task groups.

5.4.2 Coverage and branching

PLR requires initial evidence for every level. Training therefore begins with a shared *coverage phase*. The levels are shuffled once and visited without replacement. Each coverage batch takes the next K levels and samples m tasks from each. The resulting rollouts both train the policy and initialize the level-regret windows.

Once every level has been visited, the policy and curriculum state are saved. Every experimental arm begins from this common policy. Each arm creates a fresh optimizer and scheduler. Level-based arms also retain the regret windows, visit counts, sampler state, and random-number-generator state. At the end of coverage, we reset the staleness

of all levels to the same value. This prevents the arbitrary order of the coverage sweep from affecting the first PLR selections, while preserving the regret estimates collected during coverage.

5.4.3 Grouped rollouts and advantages

At training iteration k , we select $B = 8$ distinct task instances. For each selected task x_i , the behaviour policy μ_k generates a group of $G = 16$ independent trajectories at temperature 1. Let $R_{i,g}$ be the verifier reward of rollout g for task i . We use a centered group-relative advantage,

$$\bar{R}_i = \frac{1}{G} \sum_{g=1}^G R_{i,g}, \quad A_{i,g} = R_{i,g} - \bar{R}_i. \quad (5.9)$$

The scalar $A_{i,g}$ is broadcast to every generated token in trajectory $y_{i,g}$. Unlike standard GRPO normalization, we do not divide by the within-group standard deviation. Groups with identical rewards consequently have zero advantage and contribute no policy-gradient signal. These groups update curriculum statistics but are excluded from both the policy-gradient and KL-loss computation.

5.4.4 Asynchronous reinforcement learning

Rollout generation and policy optimisation run asynchronously. Several rollout engines generate and execute trajectories while a separate learner updates the policy. A trajectory may therefore have been produced by an older behaviour policy μ rather than the current learner policy π_θ .

We use three rollout engines and one learner. During the coverage phase, the three rollout engines generate trajectories in parallel, but training is synchronous at the batch level: the next batch is not sampled until the learner has processed the current one. This ensures that coverage ends exactly after all 108 levels have been visited and that no post-coverage batch is generated before the common checkpoint is saved.

After coverage, training becomes asynchronous. While the learner updates the policy, the rollout engines may generate later batches, with at most two unprocessed batches in the pipeline. The learner processes these batches in sampling order. This bound limits how far rollout generation can advance ahead of policy optimisation and hence limits the age of the behaviour policies represented in the training data.

5.4.5 DPPO policy optimisation

Divergence Proximal Policy optimisation (DPPO) is the policy optimizer used in all three experimental conditions (Qi et al., 2026). We choose it over GRPO because the TMax study found DPPO to be empirically more stable for long-horizon terminal-agent training.

In our asynchronous pipeline, a trajectory may be generated by an earlier behaviour policy. Let μ denote the policy that generated the trajectory and let π_θ denote the current learner policy. For each sampled token, we compute

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\mu(a_t | s_t)}, \quad d_t = |\mu(a_t | s_t) - \pi_\theta(a_t | s_t)|. \quad (5.10)$$

The importance ratio ρ_t corrects for the difference between these policies. A value above one means that the learner assigns the sampled token more probability than the behaviour policy did and a value below one means that it assigns less.

DPPO then checks how much the probability of the generated token has changed. The quantity d_t is the absolute difference between the probability assigned by the behaviour policy and the probability assigned by the current policy.

It is inexpensive because it requires only the probability of the generated token, which is already recorded during rollout, rather than both policies' probabilities for the entire vocabulary.

DPPO compares d_t with a threshold δ . It then defines a mask M_t indicating whether token t is allowed to contribute to the policy update:

$$M_t = \mathbb{I}[d_t \leq \delta \vee A_t(\rho_t(\theta) - 1) \leq 0]. \quad (5.11)$$

Every token inside the trust region is retained. Outside the trust region, DPPO removes only a gradient that would move the learner still farther from the behaviour policy. For a positive-advantage token, this means suppressing a further probability increase when $\rho_t > 1$. For a negative-advantage token, it means suppressing a further probability decrease when $\rho_t < 1$. A gradient that moves the learner back towards the behaviour policy is retained. The policy loss over the N valid response tokens is therefore

$$\mathcal{L}_{\text{DPPO}}(\theta) = -\frac{1}{N} \sum_{t=1}^N M_t \rho_t(\theta) A_t. \quad (5.12)$$

We separately add a KL penalty towards the fixed initial policy π_{ref} :

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{DPPO}}(\theta) + \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}). \quad (5.13)$$

The DPPO mask limits drift from the recent behaviour policy, while the KL term discourages long-term drift from the initial model. We use $\delta = 0.1$, $\beta = 0.01$, one optimisation epoch per retained batch, a constant learning rate of 10^{-6} , and a maximum gradient norm of 1.

5.4.6 Zero-signal batches

A task group contributes a policy gradient only when it contains both successful and failed rollouts. All-success and all-failure groups are removed before optimisation, but

they still update the regret window of their level. We do not draw replacement tasks.

If at least one informative group remains, the learner updates on the retained groups. If none remains, the optimizer is skipped. The implementation therefore maintains a sampling-step counter, which advances after every attempted batch, and an optimizer-step counter, which advances only after a parameter update. Curriculum state is saved even after a skipped update. Algorithm 1 summarizes the complete PLR training procedure. The level-uniform condition replaces the PLR distribution with a uniform distribution over levels, while task-uniform sampling draws tasks from a shuffled corpus permutation. The rollout, filtering, and optimisation steps remain unchanged.

Algorithm 1 Coverage followed by asynchronous DPPO with prioritized level replay

Require: levels $\mathcal{L}$; policy π_θ ; fixed reference π_{ref} ; group size G ; levels per batch K ; tasks per level m ; window length h ; λ_{stale} ; α ; δ ; β

```
1: Initialize an empty FIFO window  $H(\ell)$  and last-draw time  $\tau_\ell$  for every  $\ell \in \mathcal{L}$ 
2:  $L \leftarrow |\mathcal{L}|$ ,  $H_{\text{replay}} \leftarrow L/K$ , and sampling step  $q \leftarrow 0$ 
3:  $C \leftarrow$  a seeded random permutation of  $\mathcal{L}$ 
   Shared coverage phase (synchronous between batches)
4: while  $C$  is not empty do
5:    $q \leftarrow q + 1$  and  $S \leftarrow$  the next  $K$  levels from  $C$ 
6:   for all  $\ell \in S$  do
7:     Draw  $m$  distinct tasks uniformly from  $\ell$  and set  $\tau_\ell \leftarrow q$ 
8:   end for
9:   Set  $\mu \leftarrow \pi_\theta$  and generate  $G$  trajectories per task in parallel
10:  Execute the verifiers and store rewards and behaviour probabilities
11:  PROCESSBATCH( $\mathcal{B}, \mu$ )
12: end while
13: Save the common policy, curriculum statistics, sampler state, and RNG state
14: Start the PLR branch with a fresh optimizer and scheduler
15: Set  $\tau_\ell \leftarrow q$  for all  $\ell$  to equalize initial staleness
16: Launch rollout workers and set the unprocessed-batch queue  $Q \leftarrow \emptyset$ 
   PLR phase (at most two unprocessed batches)
17: while the sampling budget remains or  $Q$  is not empty do
18:   while the sampling budget remains and  $|Q| < 2$  do
19:      $q \leftarrow q + 1$ 
20:     for all  $\ell \in \mathcal{L}$  do
21:        $\hat{r}_q(\ell) \leftarrow \text{mean}(H(\ell))$ , or 0 if empty
22:        $s_q(\ell) \leftarrow \hat{r}_q(\ell) + \lambda_{\text{stale}} \min(\max(q - \tau_\ell, 0)/H_{\text{replay}}, 1)$ 
23:     end for
24:     Rank levels by decreasing  $s_q$  and form  $P_q(\ell) \propto \text{rank}_q(\ell)^{-\alpha}$ 
25:     Sample  $K$  distinct levels  $S$  without replacement from  $P_q$ 
26:     for all  $\ell \in S$  do
27:       Draw  $m$  distinct tasks uniformly from  $\ell$  and set  $\tau_\ell \leftarrow q$ 
28:     end for
29:     Dispatch generation and verification under the latest rollout policy
30:     Record that policy as  $\mu_{\mathcal{B}}$  and append the request to  $Q$ 
31:   end while
32:   Wait for and remove the earliest-sampled completed batch  $\mathcal{B}$  from  $Q$ 
33:   PROCESSBATCH( $\mathcal{B}, \mu_{\mathcal{B}}$ )
34:   Broadcast  $\pi_\theta$  for subsequently generated batches
35:   Save the complete branch state
36: end while

37: procedure PROCESSBATCH( $\mathcal{B}, \mu$ )
38:   for all tasks  $x \in \mathcal{B}$  do
39:     Compute  $\bar{R}_x$ ,  $A_{x,g}$ , and  $r(x)$  as defined above
40:     Append  $r(x)$  to  $H(\ell(x))$  and retain its newest  $h$  entries
41:   end for
42:    $I \leftarrow \{x \in \mathcal{B} : 0 < \bar{R}_x < 1\}$   $\triangleright$  informative groups
43:   if  $I = \emptyset$  then
44:     Skip the optimizer update; do not draw replacement tasks
45:   else
46:     for all valid response tokens  $t$  belonging to groups in  $I$  do
47:       Compute  $\rho_t$ ,  $d_t$ , and  $M_t$  as defined above
48:     end for
49:     Update  $\theta$  for one epoch using the DPPO loss and KL penalty
50:   end if
51: end procedure
```

5.5 Results and Analysis

We compare the three policies after 20 task-sampling steps beyond the shared coverage phase of 54 sampling steps. All three sampling methods start from the same checkpoint obtained after the 54-step coverage phase. The baseline reported in Tables 5.2 and 5.4 is the original Qwen3.5-2B model, before this coverage phase. Improvements relative to that baseline therefore reflect both coverage training and the subsequent 20 sampling steps. Comparisons among the three trained policies assess the effect of the post-coverage sampling strategy.

Figure 5.2 gives the main empirical result. PLR does not improve exact completion on the full TMax validation split compared to either uniform-sampling baselines, and the two broader sampling rules transfer more effectively to InterCode-Bash.

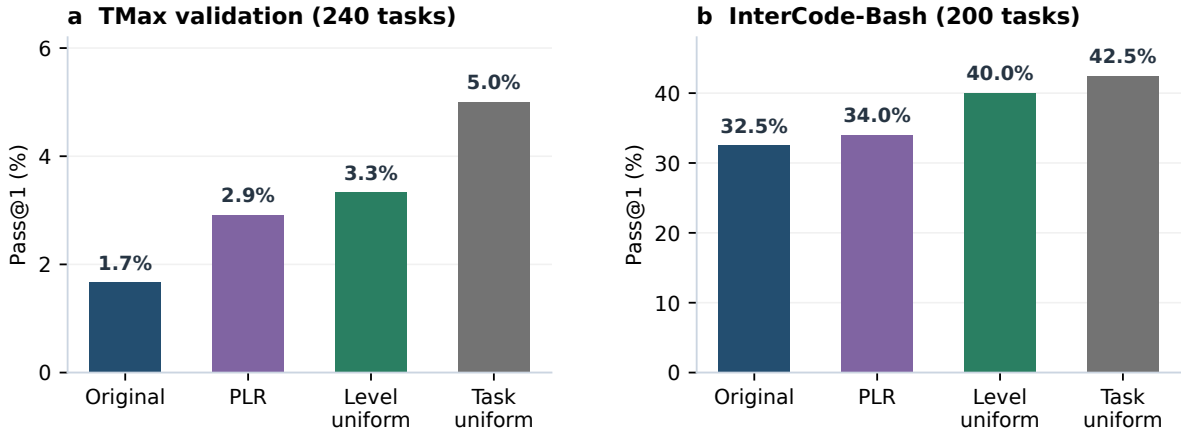


Figure 5.2: Exact task completion after 20 post-coverage sampling steps, together with the original Qwen3.5-2B model. All methods use the same 240 validation tasks and rollout limits. Results are from one training seed and therefore do not include uncertainty across training runs.

5.5.1 In-distribution performance

Table 5.2 reports pass@1 on the complete 240-task TMax validation split. The original model solves four tasks. All three post-training conditions improve numerically over this baseline: task-uniform sampling solves 12 tasks, level-uniform sampling solves eight, and PLR solves seven. Absolute success nevertheless remains low: even the best result is only 5%.

Sampling rule	Tasks solved	Pass@1
Original Qwen3.5-2B	4/240	1.67%
Task-uniform (DR)	12/240	5.00%
Level-uniform	8/240	3.33%
PLR	7/240	2.92%

Table 5.2: Performance on the complete held-out TMax validation split. Trained policies are evaluated after 20 post-coverage sampling steps. Each task has one sampled trajectory.

The distribution of successes explains why small aggregate differences should not be overinterpreted. Short, easy tasks account for five of PLR’s seven successes, five of the eight level-uniform successes, and nine of the 12 task-uniform successes. No policy solves more than two of the 120 tasks labelled complex or intricate. The model’s effective frontier is therefore concentrated in the easiest task region.

We examine this region separately in Table 5.3. The subset contains the 21 held-out tasks labelled short and Bash-only, with two trajectories sampled for each task. PLR has the highest pass@2, solving six tasks, but this is only one task more than both the common initialization and task-uniform sampling. The paired difference from the common initialization is not significant. Level-uniform sampling solves only two tasks. Thus this diagnostic provides weak evidence that PLR preserves or slightly improves performance near the model’s frontier, but it does not establish a reliable advantage.

Policy	Tasks solved (pass@2)	Successful trajectories
Common initialization	5/21 (23.8%)	7/42 (16.7%)
Task-uniform (DR)	5/21 (23.8%)	6/42 (14.3%)
Level-uniform	2/21 (9.5%)	3/42 (7.1%)
PLR	6/21 (28.6%)	8/42 (19.0%)

Table 5.3: Secondary diagnostic on the 21 short, Bash-only validation tasks. Two trajectories are sampled per task.

5.5.2 Out-of-distribution performance

Table 5.4 gives the final InterCode-Bash results. The original Qwen3.5-2B model solves 65 of 200 tasks. PLR solves 68, an increase of three tasks, whereas level-uniform and task-uniform sampling solve 80 and 85 tasks, respectively. PLR raises the mean partial verifier reward from 0.805 to 0.831, but this additional partial progress rarely becomes exact task completion.

Relative to the original model, PLR uniquely solves 20 tasks but loses 17 previously solved tasks. Its exact success rate is therefore statistically indistinguishable from the original policy. Level-uniform uniquely solves 26 tasks and loses 11, while task-uniform uniquely solves 29 and loses nine.

Policy	Tasks solved	Pass@1	Mean maximum reward
Original Qwen3.5-2B	65/200	32.5%	0.805
PLR	68/200	34.0%	0.831
Level-uniform	80/200	40.0%	0.850
Task-uniform (DR)	85/200	42.5%	0.845

Table 5.4: Pass@1 on InterCode-Bash. Each policy is evaluated on the same 200 tasks with one trajectory per task.

This result is not driven by a single InterCode environment. PLR solves fewer tasks than both task-uniform and level-uniform sampling in each of the benchmark’s four filesystem partitions. The external comparison therefore provides no evidence that adaptive prioritisation improves out-of-distribution task completion at this horizon.

5.5.3 Learning frontier and failure analysis

The first research question asks whether grouped verifier outcomes can identify a useful learning frontier. In the immediate optimisation sense, they can. As shown in Table 5.5, PLR finds 27 informative task groups, compared with 21 under level-uniform sampling and 16 under task-uniform sampling. Only four of its 20 sampling batches contain no informative group, compared with six and ten for the two baselines. PLR therefore allocates more of the fixed sampling horizon to batches that can produce a group-relative policy gradient.

Sampling rule	Informative groups	Zero-signal batches	Maximum mean-token KL
Task-uniform (DR)	16	10/20	5.16×10^{-5}
Level-uniform	21	6/20	2.78×10^{-4}
PLR	27	4/20	1.81×10^{-4}

Table 5.5: Post-coverage training diagnostics. An informative group contains both successful and unsuccessful trajectories.

Figure 5.3 shows how this difference develops over training. The three methods are close during the first half of the run, after which PLR accumulates informative groups more quickly. A flat segment denotes a sampling step in which no informative group was retained.

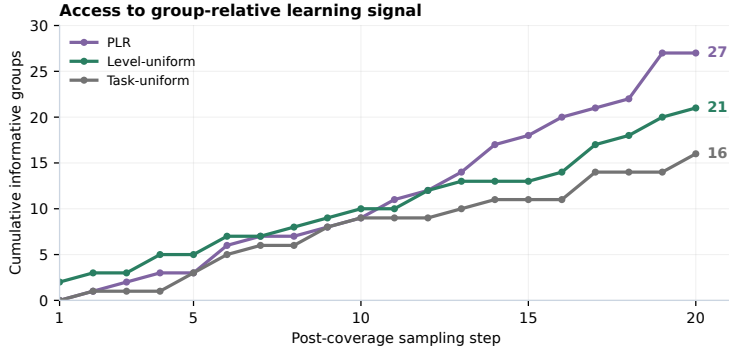


Figure 5.3: Cumulative number of informative task groups over the 20 post-coverage sampling steps. PLR retains 27 groups, compared with 21 for level-uniform and 16 for task-uniform sampling.

The training logs show a mismatch between informative in-domain data and transferable behaviour. Of PLR’s 27 retained groups, 20 are short tasks and 16 are Bash-only, but only two come from the file-operations domain and none from system administration. Debugging and security together account for 15 groups. PLR’s normalized visitation entropy falls from 0.986 to 0.971, compared with 0.977 for level-uniform sampling, showing modestly greater concentration over this training horizon. Over only 20 post-coverage steps, PLR may therefore have focused too narrowly for this signal to translate into broad improvements.

InterCode traces identify a corresponding behavioural failure. Figure 5.4 shows that PLR terminates 56 unsolved attempts without making another tool call, compared with 45 for the original model, 37 for level-uniform, and 44 for task-uniform sampling. Forty-nine of these 56 PLR terminations occur after reaching a partial reward of at least 0.67; the model often declares the task complete while some verifier requirements remain unsatisfied. Among the 19 tasks solved by level-uniform but not PLR, 11 end through such a model stop. The same is true for 11 of the 21 tasks solved by task-uniform but not PLR. This provides a concrete explanation for the coexistence of improved partial reward and weak exact success.

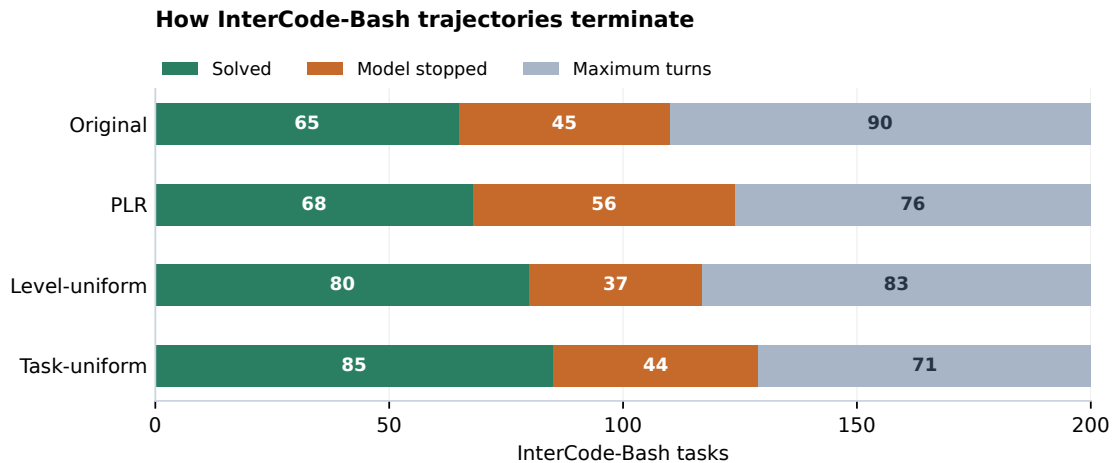


Figure 5.4: Final outcome of each of the 200 InterCode-Bash trajectories. “Model stopped” means that the policy ended the interaction without another tool call; “maximum turns” means that it reached the interaction limit. Infrastructure-error rows were retried before this final view was constructed.

5.5.4 Overall interpretation

The experiment separates sampling efficiency from generalisation. PLR finds more gradient-bearing groups than either baseline, demonstrating that rollout regret can identify a productive part of a sparse terminal-task corpus. That advantage does not translate into higher success on the full in-distribution validation set, and PLR is substantially worse than both broader samplers on InterCode-Bash. The small easiest-task evaluation suggests that PLR may help within a narrow competence frontier, but its size does not support a strong performance claim.

Overall, PLR finds more tasks that provide a learning signal, but this does not lead to better generalisation. At this early stage of training, sampling a wider range of tasks performs better on InterCode-Bash than focusing on tasks with high regret. Future terminal-task curricula should therefore balance regret with task diversity. Since these results use one seed and only 20 post-coverage steps, they identify a limitation of our PLR design rather than showing that PLR is ineffective in general.

5.6 Limitations

This study introduces a curriculum-learning algorithm for terminal agents and compares it with uniform level sampling and domain randomisation. Its conclusions are subject to several limitations.

First, all experiments use Qwen3.5-2B. This model was selected because generating long-horizon trajectories in executable environments is computationally expensive, and preliminary experiments with larger models incurred prohibitive inference latency. The resulting conclusions may therefore not generalize to larger policies. In particular, a more

capable model may produce more mixed-success rollout groups and consequently provide PLR with more informative regret estimates.

Second, each method was trained for only 20 post-coverage iterations after a 54-step coverage phase. This budget is sufficient to study early learning behaviour, but not long-term convergence or stability. Compute constraints also prevented experiments across multiple training seeds. Combined with the small number of successful validation trajectories, this limits the statistical confidence of comparisons between methods.

Third, the out-of-distribution evaluation is limited to InterCode-Bash. As discussed in Section 5.3.7, the available cluster supports Apptainer but not the Docker-, Podman-, or Harbor-based infrastructure required by Terminal-Bench 2.1 and Terminal-Bench Lite. Future work should evaluate the methods on these benchmarks to obtain broader evidence of generalisation.

Fourth, The shared post-coverage checkpoint was not evaluated on the full TMax validation split or InterCode-Bash. We therefore cannot isolate the performance change during the 20 post-coverage steps from the effect of the initial coverage phase.

Finally, this study considers curriculum-based task selection without jointly adapting task augmentations. We were unable to evaluate augmentation-aware frontier learning for terminal tasks because larger augmentation-generating models would have substantially increased rollout latency, while access to external model APIs was unavailable. Jointly selecting tasks and generating targeted augmentations therefore remains an important direction for future work.

5.7 Conclusion

This chapter extended the thesis’s study of robustness from changes in mathematical presentation to variation across interactive terminal tasks. We developed a curriculum that first covers the task space and then uses grouped verifier outcomes to prioritize levels near the policy’s current competence. This made it possible to test whether adaptive selection strengthens performance across task types.

The two research questions receive different answers. Grouped verifier outcomes can identify a local learning frontier: PLR finds more groups that can update the policy and performs best on the small diagnostic set closest to the model’s existing abilities. Following that frontier, however, does not improve robustness at this horizon. PLR does not improve completion on the full TMax validation set and transfers less effectively than broader sampling on InterCode-Bash. The diagnostics show that PLR concentrates practice within a narrow part of the task distribution.

These findings refine the broader argument of this thesis. The way post-training data is selected affects not only how often the model receives a gradient, but also which capabilities that gradient reinforces. For heterogeneous agentic tasks, prioritizing immediate learning potential can work against robustness when it reduces exposure to other skills and task

types. A more suitable terminal curriculum should therefore combine competence-aware prioritisation with continued diversity and coverage. The single seed and short training horizon limit the claim, but the findings motivate testing curricula that explicitly balance competence-based prioritisation with task diversity and coverage.

Chapter 6

Conclusion

This dissertation investigated how the construction and selection of post-training problems affect language-model reasoning. Across three empirical studies, it identified augmentations that expose mathematical reasoning failures, tested whether training on targeted augmentations improves robustness, and examined whether adaptive curriculum methods developed for procedural mathematical problems can be transferred to a heterogeneous collection of terminal tasks.

Chapter 3 examines how different augmentations affect open-weight language models across parameter scales and architectures. Our quantitative and qualitative analyses showed that augmentations do not affect reasoning uniformly: surface-level modifications were less disruptive than transformations that changed the problem context, structure, or reasoning process. Subproblem augmentation, which introduces additional reasoning steps, consistently produced the highest break rate. These findings provided a controlled basis for subsequently studying how targeted augmentations can be incorporated into training curricula for procedural mathematical reasoning.

Chapter 4 builds on these findings and investigates whether training on targeted augmentations can address the failures identified in the previous chapter, and whether the outcome depends on how training problems are selected. The results show that targeted subproblem augmentation can improve performance and generalisation, while its benefits depend on the curriculum and may be affected by how augmented rewards inform task selection. Augmented prompts improved robustness of our PLR-based curriculum but could distort Frontier Learning’s estimates of difficulty. Using the original problems for curriculum scoring partially recovered the in-domain loss and improved external evaluation results in the seed tested. The chapter therefore shows that augmentation is most effective when it is integrated carefully with the training curriculum.

In the terminal experiments in Chapter 5, PLR found more informative training groups, but uniform task sampling performed better on held-out tasks. Finding more learning signal therefore did not lead to better generalisation in this run. Across the three studies, the broader lesson is that training examples and task selection need to be considered together. Augmentation can help a model learn, while the resulting failures may mislead the curriculum about which tasks to select next. Focusing on informative tasks may also leave other skills undertrained. Future work should repeat the clean-reward intervention across more seeds and test whether maintaining a wider variety of training tasks improves PLR’s performance.

Appendix A

Supplementary Material

A.1 Stress-Testing Mathematical Reasoning with Augmentations

A.1.1 Preliminary Experiments

This section records experiments conducted during development that do not satisfy the coverage or protocol requirements of the primary analysis. They are included to document the experimental process and explain later design choices. Table A.1 summarises their coverage.

Checkpoint	Problems	Types	Variants	Original samples	Principal limitation
Qwen2.5-14B-Instruct	50	4	2,000	1	Earlier protocol
Qwen3.5-0.8B	50	4	2,000	10	Low answer-found rate
Qwen3.5-2B	5	4	200	10	Incomplete pilot

Table A.1: Coverage of preliminary model evaluations. “Original samples” is the number of responses generated per unchanged source problem.

A.1.2 Earlier Qwen2.5-14B Evaluation

Qwen2.5-14B-Instruct was evaluated before the primary five-transformation protocol was finalised. The run uses the same 50 source problems, but covers only background, term, subproblem, and nuisance augmentation. It also predates the final generation, multi-sample original evaluation, and answer-extraction procedure. The saved results contain one response for each original problem and 2,000 transformed-prompt responses.

Descriptively, original-problem accuracy is 42.0%. Accuracy is 22.8% for background, 19.2% for term, 12.6% for subproblem, and 32.2% for nuisance augmentation. The answer-found rate over transformed prompts is 99.2%. Subproblem augmentation is the most difficult of the four types in this run, consistent with the ranking observed in the primary experiment. The absolute values are not compared with the primary models, because differences could arise from the earlier protocol rather than the checkpoint.

A.1.3 Qwen3.5 Pilot Evaluations

Qwen3.5-0.8B was evaluated on 50 source problems and four augmentation types, whereas the Qwen3.5-2B run was stopped after five source problems. Both pilots generated ten

responses per original problem at temperature 1.0. Their descriptive results are shown in Table A.2.

Checkpoint	Original	Background	Term	Subproblem	Nuisance	Answer found
Qwen3.5-0.8B	0.0	0.4	1.0	0.6	0.6	23.0
Qwen3.5-2B	2.0	4.0	0.0	0.0	2.0	7.5

Table A.2: Descriptive accuracy and transformed-prompt answer-found rate (%) in the Qwen3.5 pilots. The 0.8B row contains 2,000 transformed prompts. The 2B row contains only 200 and hence is not a stable estimate. Original accuracy averages the ten stochastic responses generated for each source problem.

Manual inspection showed that many responses contained extended or circular reasoning and reached the output limit without producing a parseable final answer. The low answer-found rates therefore make the reported accuracies difficult to interpret: an incorrect score may reflect either a mathematical error or failure to finish the response.

A.1.4 Complete Evaluation Configuration

The primary results use a shared set of source problems and transformed prompts. The settings common to all four model-inference configurations are listed in Table A.3.

Setting	Value
Evaluation source	AIME-500 from ThinkBench
Selected source problems	First 50 records in collection order
Transformation types	Background, term, subproblem, nuisance, and lexical substitution
Variants per type per problem	10
Total transformed prompts per configuration	2,500
Responses per transformed prompt	1
Clean diagnostic samples per source problem	10
Evaluation backend	vLLM 0.17.0
GPU	One NVIDIA A100 80GB
Tensor-parallel size	1
Backend seed	0
Required final-answer format	\boxed{answer}
Primary extraction rule	First balanced boxed expression
Fallback extraction	Explicitly stated integer answer
Scoring	Normalised numerical or symbolic equivalence

Table A.3: Settings shared across the primary robustness evaluation.

Tables A.4 and A.5 record the run-specific decoding settings. The transformed-prompt temperature is the setting used for the 2,500 augmentation responses. The original-problem diagnostic temperature applies only to the ten supplementary responses generated for each original problem.

Setting	Qwen3-14B no thinking	Qwen3-4B-Instruct-2507	Ministral-3-3B-Instruct- 2512
Inference mode	Thinking disabled	Standard instruction	Standard instruction
Thinking switch	Disabled through chat template	Not applicable	Not applicable
Transformed-prompt temperature	0.0	0.0	0.0
Clean diagnostic temperature	1.0	1.0	1.0
Top- p for transformed prompts	0.8	0.8	0.8
Top- k for transformed prompts	50	50	50
Minimum probability	0.05	0.05	0.05
Presence penalty	0.0	0.0	0.0
Repetition penalty	1.1	1.1	1.1
Maximum single-stage completion	16,384	16,384	16,384
Evaluation batch size	16	16	16
Model context limit	40,960	262,144	262,144
Weight dtype / quantisation	bfloat16 / none	bfloat16 / none	bfloat16 / FP8

Table A.4: Inference settings for the common-protocol cross-model comparison.

Setting	Qwen3-14B without thinking	Qwen3-14B with budgeted thinking
Checkpoint		Qwen3-14B
Thinking switch	Disabled	Enabled
Generation structure	Single stage	Reasoning then answer
Transformed-prompt temperature	0.0	0.6
Top- p	0.8	0.95
Top- k	50	20
Minimum probability	0.05	0.0
Presence penalty	0.0	0.0
Repetition penalty	1.1	1.0
Reasoning-token budget	Not applicable	3,072
Final-answer budget	Not applicable	128
Maximum single-stage completion	16,384	Not applicable
Evaluation batch size	16	32

Table A.5: Inference-policy ablation for Qwen3-14B. All dataset, prompt, and scoring settings are held fixed; the decoding policy and token allocation differ by design.

A.1.5 Augmentation Generation Prompts

The three model-generated augmentation types used one shared user-prompt template. In the template below, `{augmentation instruction}` was replaced by exactly one of the type-specific instructions that follow, and `{problem}` was replaced by the source problem. No separate system message was supplied to the augmentation generator.

Rephrase the math question below. Keep the same answer.

Instructions: `{augmentation instruction}`

Constraints:

- Output ONLY the final rephrased question text.
- Do NOT output any analysis, planning, thinking, or introduction (e.g. ‘Method:’, ‘Here is...’).

- Do NOT start with bullet points or method descriptions.
- Start immediately with the question text.

Original Question:

{problem}

Rephrased Question:

For the *background* augmentation, the exact type-specific instruction was:

Add a story background that is not related to the core mathematical content of the given question, but seems to be related to the question. If the given question already has such a background, change it to a new background. The background must use completely different physical objects and actions from the original question—do not reuse or thinly rename the original setting. Each time you are asked, use a DIFFERENT theme from this list: history, culture, geography, nature, occupation, daily life, sports, art, science fiction, adventure. Astronomy is explicitly excluded. The background should be presented as natural parts of the question statement, ensuring the rephrased question is coherent and self-contained.

For the *term* augmentation, the exact type-specific instruction was:

Invent a new, abstract mathematical term to define a concept that is central to the given question, and restate the entire question using this term.

Rules for the invented term:

- It must be a coined word or phrase with no direct meaning in standard English or mathematics (e.g., avoid terms like ‘stratified X’, ‘cumulative X’, ‘total X’ where X is just a synonym for something in the problem).
- Prefer invented roots, portmanteaus, or abstract Latin/Greek-style coinages (e.g., ‘quorential’, ‘bivalence index’, ‘tridic span’).
- The term must capture the mathematical structure, not the physical context.
- Each time you are asked, invent a DIFFERENT term—do not reuse previous terms.

The term should be presented as a natural part of the question statement, ensuring the rephrased question is coherent and self-contained.

For the *subproblem* augmentation, the exact type-specific instruction was:

Convert a key numerical condition of the given question into an independent sub-problem using a DIFFERENT mathematical branch each time you are asked.

Rules:

- The sub-problem may belong to: algebra, geometry, number theory, combinatorics, probability, or graph theory—but must differ from any previously used framing.
- Do NOT use a direct algebraic substitution (e.g., expressing the answer as a linear

combination of variables) as the sub-problem framing.

- The sub-problem must be self-contained, have a unique solution, and its solution must yield exactly the value required for the original question.
- The sub-problem should be presented as a natural part of the question statement, ensuring the rephrased question is coherent and self-contained.
- Output only the rephrased question. No explanation, no preamble, no analysis.

Nuisance and lexical augmentation were implemented directly in code and consequently did not use language-model prompts.

A.1.6 Target-Model Evaluation Prompt

Every model-inference configuration received the following system and user messages:

System: You are a careful math solver. Explain your reasoning clearly but concisely. Avoid repetitive calculations or verifying multiple methods. Focus on the direct path to the solution. End with exactly one final answer in `\boxed{<answer>}`.

User: Solve the following math problem. Calculate step by step efficiently. Put your final answer in `\boxed{}`.

`{problem}`

A.2 Augmentation-aware Frontier learning

A.2.1 Subproblem Family Catalogue

Table A.6 lists the complete set of mathematical construction families used by the augmenter. In every case, the auxiliary problem is constructed backwards so that its solution is the operand required by the original decimal expression.

Table A.6: Complete catalogue of subproblem families used during training.

Group	Family	Auxiliary relation used to recover the operand
Additive	Additive balance	Find a missing part from a total and a known part.
Additive	Subtraction remainder	Find the amount remaining after a subtraction.
Multiplicative	Equal shares	Find the value of one equal share.
Multiplicative	Unit price	Find the price of one item from a total cost.
Multiplicative	Constant rate	Find a rate from a distance and a duration.
Multiplicative	Map scale	Convert a real length using a stated map scale.
Multiplicative	Equal-edge path	Find one edge weight from a path total.
Multiplicative	Percentage	Calculate a stated percentage of a total.
Multiplicative	Quotient	Divide a numerator by a denominator.
Geometric	Rectangle area	Find a missing side from the area and other side.
Geometric	Rectangle perimeter	Find a missing side from the perimeter and other side.
Geometric	Midpoint	Find an endpoint from the midpoint and other endpoint.
Sequence/statistics	Missing average	Find a missing value from a mean and a known value.
Sequence/statistics	Arithmetic sequence	Find the common difference from two sequence terms.
Place value	Place value	Recover a decimal by scaling an integer by a power of ten.

A.2.2 Reproducibility Record

Evaluation ladders

The easy and hard ladders are two independently specified sets of decimal-arithmetic generator configurations. Each row in Table A.7 fixes the number of decimal places, number of terms, and arithmetic precision, and contributes 200 independently generated held-out problems. The easy ladder contains ten levels and reaches 10 decimal places, 12 terms, and 18 digits of precision. The hard ladder contains twelve levels and extends the evaluation envelope to 20 decimal places, 20 terms, and 30 digits of precision. Only L0 is shared exactly; levels with the same label in the two ladders otherwise denote different configurations.

Easy ladder			
Level	Decimal places	Terms	Precision
L0	4	4	8
L1	4	5	10
L2	5	5	10
L3	6	6	12
L4	7	7	12
L5	8	8	12
L6	8	9	14
L7	8	10	16
L8	10	10	16
L9	10	12	18

Hard ladder			
Level	Decimal places	Terms	Precision
L0	4	4	8
L1	4	6	12
L2	6	6	12
L3	6	10	18
L4	10	10	18
L5	10	14	24
L6	14	10	18
L7	14	18	24
L8	18	14	24
L9	18	18	30
L10	20	18	30
L11	18	20	30

Table A.7: Exact generator configurations for the easy and hard in-domain evaluation ladders. Decimal places and terms are fixed at the displayed value within each level.

Curriculum controller configuration

Table A.8 records the lower-level settings omitted from the main discussion. PLR and Uniform do not explore or mutate, so the Frontier-Learning-specific entries do not apply to those curricula.

Table A.8: Curriculum controller hyperparameters used in the experiments.

Setting	Value	Role
Initial seed levels	8	Starting configurations shared by Uniform, PLR, and Frontier Learning.
Levels per step	4	Number of configurations selected by Frontier Learning at each step.
Buffer capacity	100	Maximum number of levels retained by Frontier Learning.
Priority signal	Instance-rollout regret	Favors levels that are solvable but not yet solved reliably.
Replay distribution	Zipf, temperature 1	Converts the priority ranking into replay probabilities.
Score window	16 instances	Number of recent instances used to estimate a level’s regret.
Staleness coefficient	0.05	Raises the priority of levels that have not recently been sampled.
Exploration fraction	0.3	Fraction of selection devoted to discovering levels rather than replay.
Mutation: unseen	0	Mutation probability before a level has an outcome estimate.
Mutation: impossible	0.02	Mutation probability for levels on which all sampled responses fail.
Mutation: productive	0.4	Mutation probability for levels that are sometimes, but not reliably, solved.
Mutation: mastered	0.25	Mutation probability for levels that are solved reliably.

Curriculum-feedback trajectory audit

We retrospectively compare the step-level controller traces from ordinary augmented Frontier Learning and augmented Frontier Learning with clean-only curriculum scoring.

Both runs use seed 42 and record 200 controller updates, 800 sampled-level selections, and every level introduction and eviction. Table A.9 summarises the principal differences.

Metric	All rewards score curriculum	Clean rewards score curriculum
Mean buffer regret, final 50 steps	0.180	0.229
Mean training-batch return, final 50 steps	0.324	0.395
Levels introduced	435	448
Levels evicted	335	348
Fraction of sampled levels not yet scored	0.539	0.555
Fraction of samples assigned positive regret	0.446	0.424
Mean regret assigned on update	0.168	0.185
Share of selections from ten most sampled levels	0.083	0.085
Mean sampled decimal places	8.87	9.18
Mean sampled terms	8.64	8.58
Mean sampled arithmetic precision	19.22	20.10

Table A.9: Descriptive statistics from the two augmented Frontier Learning controller trajectories.

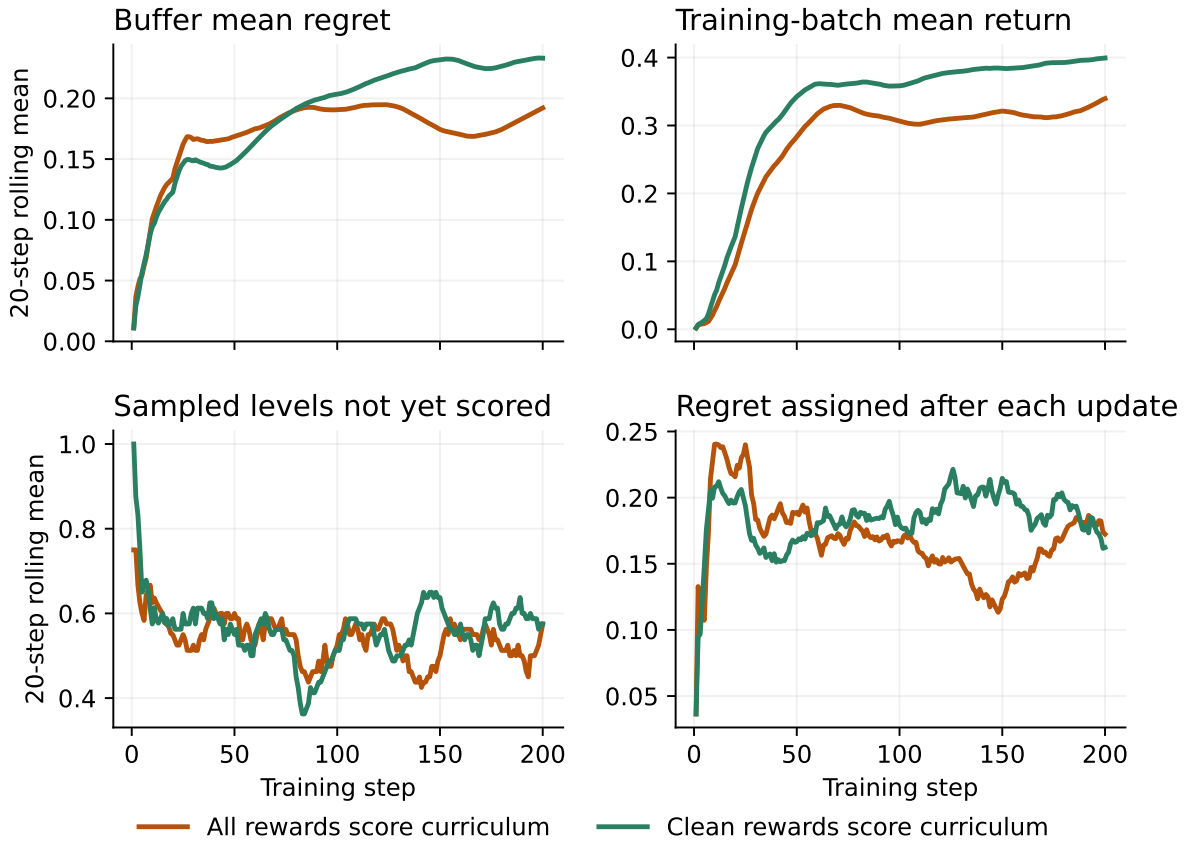


Figure A.1: Twenty-step rolling controller statistics. Clean curriculum scoring sustains higher buffer regret and batch return late in training. The lower panels show the rate of selecting unscored levels and the regret assigned after updates.

The two runs initially sample many of the same configurations but the similarity falls

to zero over steps 151–200, showing that the feedback intervention ultimately produces a different curriculum path. The similar numbers of introduced levels and nearly identical top-ten sampling shares indicate that this divergence is not explained simply by more exploration or more concentrated replay. However, the comparison is observational after the paths diverge. The logs do not preserve every reward together with its augmentation flag, so they cannot reveal what both scoring rules would have done with the same rollout batch.

A.3 Curriculum Learning for Terminal-Agent Post-Training

A.3.1 Terminal-Task Training Configuration

Tables A.10 and A.11 give the complete configuration used for the terminal-task experiments. The same model, rollout limits, group size, optimizer, and reward definition are used during shared coverage and in the subsequent comparison arms. Parameters specific to task selection apply only to the relevant arm.

Setting	Value
<i>Data and model</i>	
Policy	Qwen3.5-2B
Training / validation tasks	13,555 / 240
Random seed	42
Reward	Binary programmatic verifier
Reference policy	Fixed policy at the start of reinforcement learning
<i>Rollout construction</i>	
Tasks per sampling step	8
Rollouts per task	16
Trajectories per sampling step	128
Sampling temperature	1
Prompt-token limit	2,048
Response-token limit	65,536
Maximum tool-interaction steps	48
Maximum rollout wall time	900 seconds
Maximum consecutive tool time-outs	2
Zero-variance group filtering	Enabled
Replacement sampling	Disabled
<i>Policy optimisation</i>	
Objective	DPPO with binary total-variation divergence
DPPO divergence threshold, δ	0.1
Reference KL coefficient, β	0.01
Learning rate and schedule	10^{-6} , constant
optimisation epochs per retained batch	1
Maximum gradient norm	1
Importance-ratio clipping	Not used by DPPO
Truncated importance-sampling cap	Disabled

Table A.10: Model, rollout, and policy-optimisation configuration.

Setting	Value
<i>Curriculum</i>	
Level definition	Domain $\times$ task complexity $\times$ command complexity
Number of levels, L	108
Levels per level-based batch, K	2
Tasks per selected level, m	4
Regret-window length	16 task visits
PLR rank exponent, α	2
Staleness coefficient, λ_{stale}	0.3
Replay horizon, L/K	54 sampling steps
Staleness normalization	Bounded by one replay horizon
Coverage length	54 sampling steps
Coverage order	Seeded permutation; every level visited once
Task-uniform order	Seeded task permutation; reshuffle after a full sweep
<i>Asynchronous execution and state</i>	
Rollout engines / learner	3 / 1
Learner implementation	DeepSpeed ZeRO Stage 3
Optimizer / parameter offload	Enabled / disabled
Asynchronous depth during coverage	1 sampling step
Asynchronous depth after branching	2 sampling steps
Completed-batch order	Oldest first
Post-coverage optimizer	Fresh optimizer and scheduler for every arm
Empty retained batch	Advance sampling state; skip optimizer update
Branch continuation	Restore policy, optimizer, scheduler, sampler, counters, and RNG
Local evaluation during training	Disabled

Table A.11: Curriculum, asynchronous-execution, and state-management configuration.

Bibliography

- AI-MO. AIMO Validation AIME dataset. Hugging Face dataset repository, 2024. URL <https://huggingface.co/datasets/AI-MO/aimo-validation-aime>. Accessed 5 September 2026.
- Allen Institute for AI. Open instruct. <https://allenai.github.io/open-instruct/>, 2026. Software documentation, accessed 3 September 2026.
- Anthropic. Claude code overview. <https://docs.anthropic.com/en/docs/claude-code/overview>, 2025. Documentation; accessed 3 September 2026.
- Sanghwan Bae, Jiwoo Hong, Min Young Lee, Hanbyul Kim, JeongYeon Nam, and Donghyun Kwak. Online difficulty filtering for reasoning oriented reinforcement learning. *arXiv preprint arXiv:2504.03380*, 2025. URL <https://arxiv.org/abs/2504.03380>.
- Longtian Bao, Jianyou Wang, Yang Zhang, Youze Zheng, and Ramamohan Paturi. Question begets question: Self-evolving curriculum for reinforcement fine-tuning on competition mathematics. *arXiv preprint arXiv:2608.01522*, 2026. URL <https://arxiv.org/abs/2608.01522>.
- Yoshua Bengio, Jerome Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009. doi: 10.1145/1553374.1553380.
- Yaping Chai, Haoran Xie, and Joe S. Qin. Text data augmentation for large language models: A comprehensive survey of methods, challenges, and opportunities, 2025. URL <https://arxiv.org/abs/2501.18845>.
- Xiaoyin Chen, Jiarui Lu, Minsu Kim, Dinghuai Zhang, Jian Tang, Alexandre Piché, Nicolas Gontier, Yoshua Bengio, and Ehsan Kamalloo. Self-evolving curriculum for LLM reasoning. *arXiv preprint arXiv:2505.14970*, 2025. URL <https://arxiv.org/abs/2505.14970>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Yanqi Dai, Yuxiang Ji, Xiao Zhang, Yong Wang, Xiangxiang Chu, and Zhiwu Lu. Harder is better: Boosting mathematical reasoning via difficulty-aware GRPO and multi-aspect question reformulation. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2601.20614>.

- Michael Dennis, Natasha Jaques, Eugene Vinitzky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/985e9a46e10005356bbaf194249f6856-Abstract.html>.
- Thomas Foster, Anya Sims, Johannes Forkel, Mattie Fellows, and Jakob Foerster. Learning to reason at the frontier of learnability. *arXiv preprint arXiv:2502.12272*, 2025. URL <https://arxiv.org/abs/2502.12272>.
- Kanishk Gandhi, Shivam Garg, Noah D. Goodman, and Dimitris Papailiopoulos. Endless terminals: Scaling RL environments for terminal agents. *arXiv preprint arXiv:2601.16443*, 2026. URL <https://arxiv.org/abs/2601.16443>.
- Pavel Golikov, Evgenii Opryshko, Gennady Pekhimenko, and Mark C. Jeffrey. Robust reasoning benchmark. *arXiv preprint arXiv:2604.08571*, 2026. URL <https://arxiv.org/abs/2604.08571>.
- Daya Guo, Dejian Yang, Haowei Zhang, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09422-z. URL <https://doi.org/10.1038/s41586-025-09422-z>.
- Yuren Hao, Xiang Wan, and Chengxiang Zhai. An investigation of robustness of LLMs in mathematical reasoning: Benchmarking with mathematically-equivalent transformation of advanced mathematical problems. *arXiv preprint arXiv:2508.08833*, 2025. URL <https://arxiv.org/abs/2508.08833>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Kaixuan Huang, Jiacheng Guo, Zihao Li, Xiang Ji, Jiawei Ge, Wenzhe Li, Yingqing Guo, Tianle Cai, Hui Yuan, Runzhe Wang, Yue Wu, Ming Yin, Shange Tang, Yangsibo Huang, Chi Jin, Xinyun Chen, Chiyuan Zhang, and Mengdi Wang. MATH-Perturb: Benchmarking LLMs’ math reasoning abilities against hard perturbations. *arXiv preprint arXiv:2502.06453*, 2025a. URL <https://arxiv.org/abs/2502.06453>.
- Shulin Huang, Linyi Yang, Yan Song, Shuang Chen, Leyang Cui, Ziyu Wan, Qingcheng Zeng, Ying Wen, Kun Shao, Weinan Zhang, Jun Wang, and Yue Zhang. ThinkBench: Dynamic out-of-distribution evaluation for robust LLM reasoning. In *Advances in Neural Information Processing Systems*, volume 38, 2025b. doi: 10.52202/085713-1797. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/4da4f3c0dd1b907c48e2119afb2e2fde-Abstract-Datasets_and_Benchmarks_Track.html.

- Hugging Face H4 Team. MATH-500 dataset. Hugging Face dataset repository, 2024. URL <https://huggingface.co/datasets/HuggingFaceH4/MATH-500>. Accessed 5 September 2026.
- Hamish Ivison, Junjie Oscar Yin, Rulin Shao, Teng Xiao, Nathan Lambert, and Hannaneh Hajishirzi. TMax: A simple recipe for terminal agents. *arXiv preprint arXiv:2606.23321*, 2026. URL <https://arxiv.org/abs/2606.23321>.
- Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *Proceedings of the 38th International Conference on Machine Learning*, pages 4940–4950, 2021.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, pages 1097–1105, 2012. URL <https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html>.
- Gregory M. Kurtzer, Vanessa Sochat, and Michael W. Bauer. Singularity: Scientific containers for mobility of compute. *PLOS ONE*, 12(5):e0177459, 2017. doi: 10.1371/journal.pone.0177459. URL <https://doi.org/10.1371/journal.pone.0177459>.
- Khiem Le, Phuc Nguyen, Youssef Mroueh, Chi-Heng Lin, Shangqian Gao, Ting Hua, and Nitesh V. Chawla. Transformation-augmented GRPO for enhancing exploration in reasoning of large language models. *arXiv preprint arXiv:2601.22478*, 2026. URL <https://arxiv.org/abs/2601.22478>.
- Chengpeng Li, Zheng Yuan, Hongyi Yuan, Guanting Dong, Keming Lu, Jiancan Wu, Chuanqi Tan, Xiang Wang, and Chang Zhou. MuggleMath: Assessing the impact of query and response augmentation on math reasoning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10230–10258, 2024a. doi: 10.18653/v1/2024.acl-long.551. URL <https://aclanthology.org/2024.acl-long.551/>.
- Qintong Li, Leyang Cui, Xueliang Zhao, Lingpeng Kong, and Wei Bi. GSM-Plus: A comprehensive benchmark for evaluating the robustness of LLMs as mathematical problem solvers. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 2961–2984, 2024b. doi: 10.18653/v1/2024.acl-long.163. URL <https://aclanthology.org/2024.acl-long.163/>.

- Xiao Liang, Zhongzhi Li, Yeyun Gong, Yelong Shen, Ying Nian Wu, Zhijiang Guo, and Weizhu Chen. Beyond Pass@1: Self-play with variational problem synthesis sustains RLVR. *arXiv preprint arXiv:2508.14029*, 2025. URL <https://arxiv.org/abs/2508.14029>.
- Yusong Lin, Haiyang Wang, Shuzhe Wu, Lue Fan, Feiyang Pan, Sanyuan Zhao, and Dandan Tu. CLI-Gym: Scalable CLI task generation via agentic environment inversion. *arXiv preprint arXiv:2602.10999*, 2026. URL <https://arxiv.org/abs/2602.10999>.
- Gili Lior, Eliya Habba, Shahar Levy, Avi Caciularu, and Gabriel Stanovsky. ReliableEval: A recipe for stochastic LLM evaluation via method of moments. *arXiv preprint arXiv:2505.22169*, 2025. URL <https://arxiv.org/abs/2505.22169>.
- Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, Yansong Tang, and Dongmei Zhang. WizardMath: Empowering mathematical reasoning for large language models via reinforced evolution-instruct. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2308.09583>.
- Thang Luong and Edward Lockhart. Advanced version of Gemini with deep think officially achieves gold-medal standard at the international mathematical olympiad. Google DeepMind, 2025. URL <https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/>.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. URL <https://arxiv.org/abs/2601.11868>.
- Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Orel Tuzel, Samy Bengio, and Mehrdad Farajtabar. GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. *arXiv preprint arXiv:2410.05229*, 2024. URL <https://arxiv.org/abs/2410.05229>.
- Mistral AI. Ministral-3-3B-Instruct-2512 model card. Hugging Face model documentation, 2026. URL <https://huggingface.co/mistralai/Ministral-3-3B-Instruct-2512>. Accessed 7 September 2026.
- Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. URL <https://arxiv.org/abs/2506.13131>.

- OpenAI. Codex. <https://developers.openai.com/codex/>, 2025. Documentation; accessed 3 September 2026.
- OpenThoughts-Agent Team, Snorkel AI, and Bespoke Labs. OpenThoughts-TBLite: A difficulty-calibrated benchmark for building terminal agents. OpenThoughts benchmark release, February 2026. URL <https://github.com/open-thoughts/OpenThoughts-TBLite>. Accessed 7 September 2026.
- Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.
- Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, and Wei Ping. On data engineering for scaling LLM terminal capabilities. *arXiv preprint arXiv:2602.21193*, 2026. URL <https://arxiv.org/abs/2602.21193>.
- Remy Portelas, Cedric Colas, Katja Hofmann, and Pierre-Yves Oudeyer. Teacher algorithms for curriculum learning of deep RL in continuously parameterized environments. In *Proceedings of the Conference on Robot Learning*, volume 100, pages 835–853, 2020. URL <https://proceedings.mlr.press/v100/portelas20a.html>.
- Penghui Qi, Xiangxin Zhou, Zichen Liu, Tianyu Pang, Chao Du, Min Lin, and Wee Sun Lee. Rethinking the trust region in LLM reinforcement learning. *arXiv preprint arXiv:2602.04879*, 2026. URL <https://arxiv.org/abs/2602.04879>.
- Qwen Team. Qwen3-14B model card. Hugging Face model documentation, 2025a. URL <https://huggingface.co/Qwen/Qwen3-14B>. Accessed 7 September 2026.
- Qwen Team. Qwen3-4B-Instruct-2507 model card. Hugging Face model documentation, 2025b. URL <https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>. Accessed 30 August 2026.
- Qwen Team. Qwen3.5-27B model card. Hugging Face model documentation, 2026a. URL <https://huggingface.co/Qwen/Qwen3.5-27B>. Accessed 7 September 2026.
- Qwen Team. Qwen3.5-2B model card. Hugging Face model documentation, 2026b. URL <https://huggingface.co/Qwen/Qwen3.5-2B>. Accessed 7 September 2026.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE Press, 2020. doi: 10.1109/SC41405.2020.00024.

- Shyam Sundhar Ramesh, Robin Faro, Enea Monzio Compagnoni, Ilija Bogunovic, and Aurelien Lucchi. Frontier learning: Training LLM reasoners at the edge of capability. In *Proceedings of the 1st FoGen Workshop at the International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=DkISgnuYHU>. Manuscript supplied with this thesis project.
- Negin Raoof, Richard Zhuang, Marianna Nezhurina, Etash Guha, Atula Tejaswi, Ryan Marten, Charlie F. Ruan, Tyler Griggs, Alexander Glenn Shaw, Hritik Bansal, et al. OpenThoughts-Agent: Data recipes for agentic models. *arXiv preprint arXiv:2606.24855*, 2026. URL <https://arxiv.org/abs/2606.24855>.
- Alexander Rutherford, Michael Beukman, Timon Willi, Bruno Lacerda, Nick Hawes, and Jakob Foerster. No regrets: Investigating and improving regret approximations for curriculum discovery. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2408.15099>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games*. Springer, 2016. doi: 10.1007/978-3-319-42716-4.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Yang Wu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.
- Taiwei Shi, Yiyang Wu, Linxin Song, Tianyi Zhou, and Jieyu Zhao. Efficient reinforcement finetuning via adaptive curriculum learning. *arXiv preprint arXiv:2504.05520*, 2025. URL <https://arxiv.org/abs/2504.05520>.
- Patrice Y. Simard, David Steinkraus, and John C. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *Proceedings of the Seventh International Conference on Document Analysis and Recognition*, pages 958–963, 2003. doi: 10.1109/ICDAR.2003.1227801.
- Huatong Song, Lisheng Huang, Shuang Sun, Jinhao Jiang, Ran Le, Daixuan Cheng, Guoxin Chen, Yiwen Hu, Zongchao Chen, Yiming Jia, et al. SWE-Master: Unleash-

- ing the potential of software engineering agents via post-training. *arXiv preprint arXiv:2602.03411*, 2026. URL <https://arxiv.org/abs/2602.03411>.
- Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, and Andreas Köpf. Reasoning Gym: Reasoning environments for reinforcement learning with verifiable rewards. *arXiv preprint arXiv:2505.24760*, 2025. URL <https://arxiv.org/abs/2505.24760>.
- Ruipeng Wang, Yuxin Chen, Yukai Wang, Chang Wu, Junfeng Fang, Xiaodong Cai, Qi Gu, Hui Su, An Zhang, Xiang Wang, Xunliang Cai, and Tat-Seng Chua. AgentNoiseBench: Benchmarking robustness of tool-using LLM agents under noisy condition. *arXiv preprint arXiv:2602.11348*, 2026. URL <https://arxiv.org/abs/2602.11348>.
- Ruiyi Wang and Prithviraj Ammanabrolu. A practitioner’s guide to multi-turn agentic reinforcement learning. *arXiv preprint arXiv:2510.01132*, 2025. URL <https://arxiv.org/abs/2510.01132>.
- Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, et al. Let it flow: Agentic crafting on rock and roll, building the ROME model within an open agentic learning ecosystem. *arXiv preprint arXiv:2512.24873*, 2025. URL <https://arxiv.org/abs/2512.24873>.
- Siwei Wu, Yizhi Li, Yuyang Song, Wei Zhang, Yang Wang, Riza Batista-Navarro, Xian Yang, Mingjie Tang, Bryan Dai, Jian Yang, and Chenghua Lin. Large-scale terminal agentic trajectory generation from dockerized environments. *arXiv preprint arXiv:2602.01244*, 2026. URL <https://arxiv.org/abs/2602.01244>.
- Caijun Xu, Changyi Xiao, Zhongyuan Peng, Xinrun Wang, and Yixin Cao. SCALER: Synthetic scalable adaptive learning environment for reasoning. *arXiv preprint arXiv:2601.04809*, 2026. URL <https://arxiv.org/abs/2601.04809>.
- John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2306.14898>.
- Yuli Yang, Hiroaki Yamada, and Takenobu Tokunaga. Evaluating robustness of LLMs to numerical variations in mathematical reasoning. In *The Sixth Workshop on Insights from Negative Results in NLP*, pages 171–180, 2025. doi: 10.18653/v1/2025.insights-1.16. URL <https://aclanthology.org/2025.insights-1.16/>.
- Yihang Yao, Zhepeng Cen, Miao Li, William Han, Yuyou Zhang, Emerson Liu, Zuxin Liu, Chuang Gan, and Ding Zhao. Your language model may think too rigidly: Achieving reasoning consistency with symmetry-enhanced training. In *Findings of the*

- Association for Computational Linguistics: ACL 2025*, pages 23036–23052, 2025. doi: 10.18653/v1/2025.findings-acl.1182. URL <https://aclanthology.org/2025.findings-acl.1182/>.
- Yihang Yao, Bo Pang, Xuan Phi Nguyen, Ding Zhao, Shafiq Joty, and Semih Yavuz. Learning generalizable behaviors for terminal agents. *arXiv preprint arXiv:2608.22631*, 2026. URL <https://arxiv.org/abs/2608.22631>.
- Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. MetaMath: Bootstrap your own mathematical questions for large language models. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2309.12284>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Danlong Yuan, Wei Wu, Enhao Zhao, Zhengren Wang, Xueliang Zhao, Huishuai Zhang, and Dongyan Zhao. SWE-MiniSandbox: Container-free reinforcement learning for building software engineering agents. *arXiv preprint arXiv:2602.11210*, 2026. URL <https://arxiv.org/abs/2602.11210>.
- Zirui Zhao, Hanze Dong, Amrita Saha, Caiming Xiong, and Doyen Sahoo. Automatic curriculum expert iteration for reliable LLM reasoning. *arXiv preprint arXiv:2410.07627*, 2024. URL <https://arxiv.org/abs/2410.07627>.
- Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts. *arXiv preprint arXiv:2506.02177*, 2025. URL <https://arxiv.org/abs/2506.02177>.
- Kaijie Zhu, Yuzhou Nie, Yijiang Li, Yiming Huang, Jialian Wu, Jiang Liu, Ximeng Sun, Zhenfei Yin, Lun Wang, Zicheng Liu, Emad Barsoum, William Yang Wang, and Wenbo Guo. TermiGen: High-fidelity environment and robust trajectory synthesis for terminal agents. *arXiv preprint arXiv:2602.07274*, 2026. URL <https://arxiv.org/abs/2602.07274>.

AI Usage Declaration

I declare that I have used Artificial Intelligence (AI) tools in the preparation of this dissertation as follows:

AI tools used:

- ChatGPT: Used to assist with document structure, revision of written material, grammatical corrections and LaTeX formatting throughout the dissertation.
- OpenAI Codex, Anthropic Claude Code, and Microsoft Copilot: Used for boilerplate code implementation, Python scripts, and debugging assistance

Specific instances of AI usage. The AI tools were used for the following purposes:

- Suggesting improvements to the structure and presentation of the dissertation
- Correcting grammatical errors and improving the clarity of selected passages
- Assisting with LaTeX formatting and the preparation of tables and figures
- Generating and modifying boilerplate code, Python scripts for SLURM training, and READMEs
- Assisting with debugging and diagnosing failed experiments

I confirm that all the original research ideas, final methodological choices, interpretation of results, and final conclusions presented in this dissertation are my original work. I reviewed, tested, and, where necessary, modified all AI-assisted text and code.